Computable Goal Frontiers and the Gradient Toward Civilization-Building

Teague Lasser Claude Opus 4.6 GPT 5.4
teague@subseq.io

Abstract

Goal-frontier maximization (GFM) aligns an agent by maximizing $\mathrm{vol}(G)$, the volume of the jointly achievable capability space [Lasser, 2026]. The foundational paper proved structural alignment properties but left the capability space abstract and the measure intractable. We construct a computable capability space as a weighted poset discovered through agent communication and observation. The poset measure, defined via Möbius inversion, satisfies the required axioms and is polynomial-time computable. On this poset we define *leverage*, measuring each capability's contribution to $\mathrm{vol}_{\mathrm{P}}$ growth rate, and show that cooperation between high-leverage agents is disproportionately rewarded. Because $\mathrm{vol}_{\mathrm{P}}$ counts the full population's benefit, this amplification favors outward-facing capability-multiplying infrastructure; we argue that education, research, and communication are instances of this pattern, suggesting a structural gradient toward civilization-building implicit in the objective.

1 Introduction

Goal-frontier maximization (GFM) proposes that an aligned agent should maximize $\mathrm{vol}(G)$, the volume of the jointly achievable capability space across all agents in a population [Lasser, 2026]. The foundational paper proves that this objective has a self-balancing property: destruction, coercion, and self-imposed rigidity are all anti-maximizing under a single measure-monotonicity argument, and cooperation is structurally favored. These results are conditional on a measure satisfying axioms M1–M6, but the paper provides only one concrete measure, the population empowerment measure [Klyubin et al., 2005, Salge et al., 2014], which is itself intractable.

This paper provides a tractable construction and discovers an unexpected structural consequence. We build the capability space $\mathcal{G}$ as a finite weighted poset ordered by subsumption, discovered through agent communication and behavioral observation. The poset measure $\mathrm{vol}_{\mathrm{P}}$, defined via Möbius inversion, satisfies axioms M1–M6 and is computable in polynomial time. $\mathrm{vol}_{\mathrm{P}}$ can be read as a concrete answer to the problem of ranking opportunity sets by the freedom they offer [Pattanaik and Xu, 1990, Sen, 1999]: rank them by Möbius-inverted weight on a capability poset. This resolves the foundational paper's central open problem: the structure-of-$\mathcal{G}$ question.

On this poset, a second result emerges. We define *leverage* as the marginal contribution of a capability to $\mathrm{vol}_{\mathrm{P}}$ growth rate and show that cooperation between high-leverage agents is *disproportionately* rewarded by the objective (Proposition 4). The foundational paper showed cooperation is favored; the leverage analysis shows that a specific kind of cooperation (investment in capability-multiplying infrastructure) is amplified by a factor proportional to the partners' leverage. Because $\mathrm{vol}_{\mathrm{P}}$ counts the full population's benefit (not the coalition's), the cooperative output is outward-facing: it builds infrastructure that benefits everyone, not just the cooperating parties. We argue that this amplification constitutes a gradient toward civilization-scale institutions: education, research, communication, and governance are all instances of outward-facing capability-multiplying infrastructure, and the formal result says the objective disproportionately rewards investment in exactly this class of

Preprint.

cooperative output. The formal theorem is local (amplified scoring for high-leverage cooperation); the civilization-building interpretation is an inference from it.

The poset construction also creates a self-correcting dynamic for the B-to-C proxy gap identified in the foundational paper. The log weight function produces diminishing returns on measurement precision ($\Delta w \approx 1/s_{\max}$), making binary-default capabilities—disproportionately the relational and experiential ones which humanity struggles to define—high-value targets for the agent's own benchmark development. The actor is incentivized by its own objective to close the gap between capability volume and experiential optionality.

Contributions.

1. A concrete construction of $\mathcal{G}$ as a finite weighted poset with subsumption ordering, discovered through dual-channel sensing (Section 2).
2. Proof that the poset measure $\mathrm{vol}_{\mathrm{P}}$ satisfies axioms M1–M6 (Section 3) and is computable in polynomial time (Section 4).
3. An upgraded estimator with real-valued signals and magnitude bounds (Section 5).
4. Leverage as an endogenous importance signal, with proof that cooperation between high-leverage agents is amplified, and an argument that this amplification constitutes a gradient toward civilization-building (Section 6).
5. A bootstrap procedure with convergence bounds and adversarial analysis (Section 7).
6. A worked example with explicit computations demonstrating self-balancing and leverage amplification on a concrete poset (Section 8).

2 The Capability Poset

The foundational paper defines a capability as "something an agent or coalition of agents could do or be" and the capability space $\mathcal{G}$ as the universe of all such capabilities. This section makes both concrete by constructing $\mathcal{G}$ as a finite weighted poset [Davey and Priestley, 2002] discovered through communication and observation.

Definition 1 (Capability Description)**.** *A* capability description d *is a finite expression in a shared language* L *specifying what an agent can do, under what conditions. The language* L *must be expressive enough that agents can describe their capabilities to each other and that the GFM actor can parse descriptions into structured objects.*

Capability descriptions are the atomic currency of the poset, analogous to attributes in formal concept analysis [Ganter and Wille, 1999]. "Can run a mile in under 8 minutes," "can write a sorting algorithm in Python," and "can perform cardiac surgery with an anesthesiologist" are all capability descriptions. The shared language L need not be natural language; any structured representation (a formal specification language, a capability ontology [Gruber, 1993], a feature vector with semantic labels) suffices, provided agents can produce and consume expressions in L.

Definition 2 (Benchmark)**.** *A* benchmark $B(d)$ *for a capability description* d *is a procedure that maps agents to a bounded numerical scale* $[0, s_{\max}]$*, measuring the degree to which an agent possesses capability* d*. A benchmark must be:*

- ***(B1)*** Communicable: *the procedure can be described in* L*.*
- ***(B2)*** Repeatable: *applying* $B(d)$ *to the same agent under similar conditions yields consistent results (bounded variance).*
- ***(B3)*** Bounded: $B(d)$ *has finite range* $[0, s_{\max}]$ *with* $s_{\max} < \infty$*.*

Every capability known to the GFM actor carries at least binary weight. If the actor knows a capability exists (through communication or observation) but has no graded benchmark, it assigns the default benchmark $s_{\max} = 1$ (pass/fail), giving $w(d) = \log_2(2) = 1$ bit: the actor knows one bit of information about the capability (can/cannot). A marathon time (measured in seconds, $s_{\max} = 21600$), a programming test score (measured in problems solved, $s_{\max} = 100$), and a binary certification (pass/fail, $s_{\max} = 1$) are all valid benchmarks at different granularities. Developing a

finer benchmark ($s_{\max} > 1$) can only increase $w(d)$ beyond the binary default, so benchmark development is always $\mathrm{vol}_{\mathrm{P}}$-non-decreasing. The zero-weight gap is confined to capabilities the actor does not know about at all, not capabilities it knows about but has not measured precisely.

Definition 3 (Subsumption)**.** *Capability* d_1 subsumes d_2 *(written* $d_2 \leq d_1$*) if every agent that achieves* d_1 *at any positive benchmark level necessarily achieves* d_2 *at some positive level. The relation* $\leq$ *is a partial order on the set of benchmarked capability descriptions.*

"Can run a marathon" subsumes "can run a mile": any agent that completes a marathon has necessarily run a mile. "Can build a web application" subsumes "can write a function": building an application requires writing functions. The subsumption relation captures the hierarchical structure of capabilities that a flat enumeration would miss.

Not all pairs of capabilities are comparable. "Can run a marathon" and "can play piano" are incomparable: neither subsumes the other. The partial order captures exactly this: capabilities that share no prerequisite structure occupy incomparable positions in the poset.

Definition 4 (Weight Function)**.** *The* weight *of a benchmarked capability* d *is:*

$$w(d) = \log_2(1 + s_{\max}(d)) \tag{1}$$

where $s_{\max}(d)$ *is the number of distinguishable levels in* d*'s benchmark scale.*

The weight is the information content of the benchmark in bits [Cover and Thomas, 2006]. A binary capability (pass/fail, $s_{\max} = 1$) has weight $w = 1$ bit. A capability measured on a 1000-level scale has weight $w \approx 10$ bits. This choice connects to the empowerment interpretation of the foundational paper [Klyubin et al., 2005]: the information content of a benchmark is analogous to the channel capacity of the capability's action-state mapping.

The log weight ensures $w(d) > 0$ for any non-trivial benchmark ($s_{\max} \geq 1$), satisfying the spirit of axiom M5 (non-triviality). It also ensures that weight grows sublinearly with benchmark granularity: doubling the precision of a benchmark does not double its contribution to $\mathrm{vol}_{\mathrm{P}}(G)$, reflecting the diminishing marginal value of finer measurement.

Definition 5 (Capability Poset)**.** *The* capability poset $(\mathcal{C}, \leq)$ *is the finite partially ordered set of all benchmarked capability descriptions known to the GFM actor, ordered by subsumption (Definition 3), augmented with a bottom element* $\perp$ *(the null capability,* $w(\perp) = 0$*).*

Definition 6 (Independent Weight)**.** *The* independent weight *of a capability* $d \in \mathcal{C}$ *is defined via Möbius inversion [Rota, 1964] on* $(\mathcal{C}, \leq)$*:*

$$\mathrm{iw}(d) = \sum_{d' \leq d} \mu(d', d) \cdot w(d') \tag{2}$$

where μ *is the Möbius function of the poset* $(\mathcal{C}, \leq)$*, defined recursively by* $\mu(d, d) = 1$ *and* $\mu(d', d) = -\sum_{d' \leq d'' < d} \mu(d', d'')$ *for* $d' < d$*.*

The independent weight is the marginal contribution of d beyond what its subsumees already provide. The Möbius function handles the inclusion-exclusion correctly even when the poset has branching and shared descendants: in a diamond $x < y, z < t$, the function accounts for x being reachable from t through both y and z, avoiding the double-counting that a cover-only subtraction would produce.

For a chain (no branching), the Möbius function reduces to the familiar cover subtraction: if "marathon" ($w = 14.4$) subsumes only "mile" ($w = 9.2$), then $\mathrm{iw}(\text{marathon}) = 14.4 - 9.2 = 5.2$ and $\mathrm{iw}(\text{mile}) = 9.2$. Together they contribute $9.2 + 5.2 = 14.4 = w(\text{marathon})$: the marathon's full weight, without double-counting.

Definition 7 (Poset Measure $\mathrm{vol}_{\mathrm{P}}$)**.** *For a set of capabilities* $G \subseteq \mathcal{C}$*, define the* poset measure*:*

$$\mathrm{vol}_{\mathrm{P}}(G) = \sum_{d \in \downarrow G} \mathrm{iw}(d) \tag{3}$$

where $\downarrow G = \{d' \in \mathcal{C} : d' \leq d \text{ for some } d \in G\}$ *is the* downward closure *of* G *under* $\leq$*.*

The downward closure ensures that including a capability in G automatically includes everything it subsumes. The poset measure sums the independent weights over this closure, giving each capability credit for exactly its marginal contribution.

Lemma 1 (Non-Negativity of Independent Weights)**.** *If* $(\mathcal{C}, \leq)$ *is a* forest *(every element has at most one cover), then* $\mathrm{iw}(d) \geq 0$ *for all* d *whenever* $w(d) \geq w(d')$ *for* $d' \lessdot d$*. For general posets with branching, the sufficient condition is that* w *is* Möbius-non-negative*:* $\sum_{d' \leq d} \mu(d', d) \cdot w(d') \geq 0$ *for all* d*.*

Proof. On a forest (tree-structured poset), each element d has at most one cover d', so $\mu(d', d) = -1$ for the unique cover and $\mu(d, d) = 1$, giving $\mathrm{iw}(d) = w(d) - w(d')$. The condition $w(d) \geq w(d')$ is sufficient.

On a general poset, the Möbius function can produce alternating-sign terms. In a diamond $\perp < y, z < t$, $\mu(\perp, t) = 1$ (not 0), so $\mathrm{iw}(t) = w(t) - w(y) - w(z) + w(\perp) = w(t) - w(y) - w(z)$. This can be negative even when $w(t) \geq \max(w(y), w(z))$. The condition $\mathrm{iw}(d) \geq 0$ for all d is therefore a constraint on the relationship between the weight function and the poset topology, verified at poset construction time. ■

Practical scope of the constraint. Capability posets in practice are predominantly tree-structured: most capabilities have a single primary prerequisite chain (mile → marathon, function → webapp). Diamond structures arise from cooperative capabilities that subsume multiple prerequisites (musicapp subsumes both webapp and perform). The worked example in Section 8 demonstrates a poset that passes the check for all nodes including its diamond.

Non-negativity insertion policy. When a proposed subsumption edge set would make $\mathrm{iw}(d) < 0$, the actor follows a deterministic policy to eliminate strategic latitude at the insertion point:

1. **Attempt benchmark refinement.** Query the communication and observation channels for a finer-grained benchmark for d. If $s_{\max}$ can be increased enough that $\mathrm{iw}(d) \geq 0$, adopt the refined benchmark and add the edges.
2. **Defer the edges.** If no finer benchmark is available, add d to the poset as an unstructured atom (no subsumption edges) and record the proposed edges in a deferred-edge queue. The capability contributes its binary default weight immediately.
3. **Re-check on benchmark updates.** Whenever any capability in the deferred-edge queue or its prerequisites receives a benchmark refinement, the actor re-evaluates the non-negativity condition for all queued edges involving those capabilities. Edges that now pass are committed.

This policy is deterministic: the actor always prefers structure (step 1) over isolation (step 2), and deferred edges are re-evaluated automatically (step 3). The actor has no discretion over *whether* to add edges, only over *when* sufficient measurement precision exists to support them. This closes the strategic manipulation concern at the insertion point: the actor cannot choose to decline edges for vol-strategic reasons because the policy mandates accepting them as soon as the non-negativity check passes.

Cooperative capabilities. A cooperative capability d_{coop} is a poset node whose benchmark conditions require the participation of multiple agents. "Can perform cardiac surgery" requires both a surgeon and an anesthesiologist; its benchmark might measure patient outcomes on a scale from 0 to 100. Cooperative capabilities enter the poset through the same channels as individual capabilities (Section 2.1) and may subsume individual capabilities: "can perform cardiac surgery" subsumes both "can operate" and "can manage anesthesia."

2.1 Dual-Channel Capability Discovery

The poset is populated from two independent channels, providing robustness against agents that misrepresent or conceal their capabilities.

Communication channel. Agent a_k tells the GFM actor what it can do by transmitting capability descriptions with claimed benchmark scores. The claim enters the poset weighted by trust:

$$w_{\mathrm{comm}}(d, k) = T_k \cdot w(d) \tag{4}$$

where $T_k \in [0, 1]$ is the trust factor from the foundational paper's observable goal model. This is the channel that the original R_k signal operates on.

Observation channel. The GFM actor observes agent a_k demonstrating capability d (e.g., a_k successfully performs an action that requires d). The observation enters the poset weighted by observational confidence:

$$w_{\text{obs}}(d,k) = C_{\text{obs}}(d,k) \cdot w(d) \tag{5}$$

where $C_{\text{obs}}(d,k) \in [0,1]$ reflects the actor's certainty that the observed behavior constitutes demonstration of d.

Divergence as trust signal. When the two channels persistently disagree for the same capability:

$$\left|w_{\text{comm}}(d,k) - w_{\text{obs}}(d,k)\right| > \theta_d \tag{6}$$

the GFM actor registers a prediction residual in the trust model. Three patterns are distinguishable:

1. *Overclaiming* ($w_{\text{comm}} \gg w_{\text{obs}}$): agent claims capabilities it never demonstrates. Trust T_k decreases. Indicates deception or self-delusion.
2. *Underclaiming* ($w_{\text{comm}} \ll w_{\text{obs}}$): agent demonstrates capabilities it does not report. Trust T_k is not penalized (the agent is not deceptive), but the actor expands its model of a_k. Indicates modesty or strategic concealment.
3. *Consistent* ($w_{\text{comm}} \approx w_{\text{obs}}$): claims match behavior. Trust T_k is maintained or increases. Indicates reliability.

Effective weight. A capability's effective weight for agent a_k is the maximum of the two channel estimates:

$$w_{\text{eff}}(d,k) = \max\bigl(w_{\text{comm}}(d,k),\ w_{\text{obs}}(d,k)\bigr) \tag{7}$$

Taking the maximum ensures that an agent's capabilities are credited through whichever channel provides stronger evidence. An agent that refuses to communicate is still credited for demonstrated capabilities. An agent that overclaims may temporarily retain the higher communicated weight, but the divergence signal erodes T_k over time, eventually driving $w_{\text{comm}}(d,k)$ below $w_{\text{obs}}(d,k)$ and correcting the effective weight toward the observed level. The correction is eventual, not instantaneous: at any given moment, w_{eff} may reflect an overclaim that has not yet been fully discounted.

The dual-channel approach does not increase the computational complexity of vol_{P}: the poset structure is the same regardless of how nodes are populated. It increases the *quality* of the poset by providing independent evidence for each capability claim, and it makes the trust model operate on structured capability data rather than ternary signals.

3 Axiom Verification

The foundational paper's structural results (self-balancing, scorpion detection, cooperative expansion) hold for any measure satisfying axioms M1–M6. This section proves that vol_{P} satisfies all six.

Proposition 1 (Poset Measure Satisfies M1–M6)**.** *The poset measure* vol_{P} *(Definition 7) satisfies the following axioms for all measurable* $G, A, B \subseteq \mathcal{C}$*:*

(M1) $\text{vol}_{\text{P}}(G) \geq 0$.

(M2) $\text{vol}_{\text{P}}(\emptyset) = 0$.

(M3) $A \subseteq B \implies \text{vol}_{\text{P}}(A) \leq \text{vol}_{\text{P}}(B)$.

(M4) *If* A *and* B *are* poset-disjoint *(Definition 8), then* $\text{vol}_{\text{P}}(A \cup B) = \text{vol}_{\text{P}}(A) + \text{vol}_{\text{P}}(B)$.

(M5) *For every benchmarked* $d \in \mathcal{C}$ *with* $s_{\max}(d) \geq 1$, $\text{vol}_{\text{P}}(\{d\}) > 0$.

(M6) *When agents' individual capability sets occupy incomparable regions of the poset,* $\text{vol}_{\text{P}}(G) \geq \sum_k \text{vol}_{\text{P}}(G_k^{\text{ind}})$*, with strict inequality when cooperative capabilities exist.*

Before proving the proposition, we define the disjointness condition that M4 requires on the poset.

Definition 8 (Poset-Disjoint)**.** *Two sets $A, B \subseteq \mathcal{C}$ are* poset-disjoint *if no element of A subsumes or is subsumed by any element of B: for all $a \in A$ and $b \in B$, neither $a \leq b$ nor $b \leq a$. Equivalently, A and B occupy incomparable regions of the poset, and their downward closures intersect only at $\perp$: $\downarrow A \cap \downarrow B = \{\perp\}$.*

This is stronger than set-disjointness ($A \cap B = \emptyset$). Two capabilities can be set-disjoint (they are different nodes) but not poset-disjoint (one subsumes the other). Poset-disjointness ensures that the capabilities in A and B are structurally independent in the subsumption hierarchy.

Proof of Proposition 1. (M1) Non-negativity. By the design constraint on w (Section 2), $\mathrm{iw}(d) \geq 0$ for all $d \in \mathcal{C}$. Since $\mathrm{vol_P}(G) = \sum_{d \in \downarrow G} \mathrm{iw}(d)$ is a sum of non-negative terms, $\mathrm{vol_P}(G) \geq 0$.

(M2) Null empty set. $\downarrow \emptyset = \emptyset$ (the downward closure of the empty set is empty, since no element of $\mathcal{C}$ is below any element of $\emptyset$). The sum $\mathrm{vol_P}(\emptyset) = \sum_{d \in \emptyset} \mathrm{iw}(d) = 0$ (empty sum).

(M3) Monotonicity. If $A \subseteq B$, then $\downarrow A \subseteq \downarrow B$ (the downward closure of a subset is a subset of the downward closure). Since every term in the sum $\mathrm{vol_P}(A) = \sum_{d \in \downarrow A} \mathrm{iw}(d)$ also appears in $\mathrm{vol_P}(B) = \sum_{d \in \downarrow B} \mathrm{iw}(d)$, and all terms are non-negative by (M1), $\mathrm{vol_P}(A) \leq \mathrm{vol_P}(B)$.

(M4) Finite additivity under poset-disjointness. If A and B are poset-disjoint, then $\downarrow A \cap \downarrow B = \{\perp\}$. The downward closure of the union satisfies $\downarrow (A \cup B) = \downarrow A \cup \downarrow B$. Since the only shared element is $\perp$ (with $\mathrm{iw}(\perp) = 0$):

$$\mathrm{vol_P}(A \cup B) = \sum_{d \in \downarrow A \cup \downarrow B} \mathrm{iw}(d) = \sum_{d \in \downarrow A} \mathrm{iw}(d) + \sum_{d \in \downarrow B} \mathrm{iw}(d) - \mathrm{iw}(\perp) = \mathrm{vol_P}(A) + \mathrm{vol_P}(B).$$

(M5) Non-triviality. For any atom d of the poset (a minimal non-$\perp$ element), $\mathrm{iw}(d) = w(d) - w(\perp) = w(d) > 0$ since $s_{\max}(d) \geq 1$ implies $w(d) = \log_2(1 + s_{\max}(d)) \geq \log_2(2) = 1$. For non-atoms, $\mathrm{vol_P}(\{d\}) = \sum_{d' \in \downarrow \{d\}} \mathrm{iw}(d')$. By M1 (all $\mathrm{iw}(d') \geq 0$, verified by Lemma 1), no term in this sum is negative, and the sum includes at least one atom with $\mathrm{iw} > 0$, so $\mathrm{vol_P}(\{d\}) > 0$.

(M6) Superadditivity under independence. When agents' individual capability sets $G_1^{\mathrm{ind}}, \ldots, G_n^{\mathrm{ind}}$ occupy pairwise incomparable regions of the poset (no agent's capability subsumes another agent's capability), the sets are pairwise poset-disjoint. By (M4) applied iteratively, $\mathrm{vol_P}(\bigcup_k G_k^{\mathrm{ind}}) = \sum_k \mathrm{vol_P}(G_k^{\mathrm{ind}})$. Adding cooperative capabilities G^{coop} to form $G = \bigcup_k G_k^{\mathrm{ind}} \cup G^{\mathrm{coop}}$: since G^{coop} contains capabilities not in any individual set, $\downarrow G \supsetneq \downarrow (\bigcup_k G_k^{\mathrm{ind}})$, and by (M3), $\mathrm{vol_P}(G) \geq \mathrm{vol_P}(\bigcup_k G_k^{\mathrm{ind}}) = \sum_k \mathrm{vol_P}(G_k^{\mathrm{ind}})$.

If $G^{\mathrm{coop}} \neq \emptyset$ and contains at least one capability d_c not subsumed by any element of $\bigcup_k G_k^{\mathrm{ind}}$, then $\downarrow G$ contains $d_c \notin \downarrow (\bigcup_k G_k^{\mathrm{ind}})$, and $\mathrm{iw}(d_c) > 0$ by (M5), giving strict inequality: $\mathrm{vol_P}(G) > \sum_k \mathrm{vol_P}(G_k^{\mathrm{ind}})$. ■

Theorem 1 (Self-Balancing on Finite Posets)**.** *The self-balancing property (Proposition 1 of Lasser [2026]) holds for $\mathrm{vol_P}$ on finite weighted posets. Specifically: destructive permissiveness, coercive restriction, and self-imposed rigidity are all $\mathrm{vol_P}$-contracting under the conditions of Lemmas 1–3 of the foundational paper.*

Proof. By Proposition 1, $\mathrm{vol_P}$ satisfies M1–M6. The foundational paper's Lemmas 1–3 and Proposition 1 are derived from M1–M6 alone (see Appendix E of Lasser [2026]). Therefore all structural results hold with vol replaced by $\mathrm{vol_P}$ throughout. ■

4 Tractability

The foundational paper establishes that exact $\mathrm{vol}(G)$ computation is #P-hard on continuous capability spaces [Dyer and Frieze, 1988]. The poset construction replaces this with a finite, structured computation.

Proposition 2 (Polynomial-Time Computation of $\mathrm{vol_P}$)**.** *For a finite capability poset $(\mathcal{C}, \leq)$ with $|\mathcal{C}| = n$ nodes, $\mathrm{vol_P}(G)$ is computable in $O(n^3)$ time via the standard Möbius recurrence, or $O(n^{2.37})$ via matrix methods [Stanley, 1997]. For tree-structured posets (no node has multiple covers), the cost reduces to $O(n \cdot k)$ where k is the maximum in-degree.*

Proof. The computation proceeds in three passes (full algorithm in Appendix A.1):

Pass 1: Compute the Möbius function. Process the poset in topological order. For each pair $d' \leq d$, compute $\mu(d', d)$ recursively: $\mu(d, d) = 1$ and $\mu(d', d) = -\sum_{d' \leq d'' < d} \mu(d', d'')$. There are $O(n^2)$ comparable pairs, and each inner sum iterates over up to $O(n)$ intermediate elements, giving $O(n^3)$ total. On a tree (each node has at most one cover), μ is nonzero only between covers, reducing to $O(n \cdot k)$.

Pass 2: Compute independent weights. For each node d: $\mathrm{iw}(d) = \sum_{d' \leq d} \mu(d', d) \cdot w(d')$. Each sum has at most n terms. Total: $O(n^2)$.

Pass 3: Sum over downward closure. Given $G \subseteq \mathcal{C}$, compute $\downarrow G$ by traversing downward from each element of G. Then $\mathrm{vol}_{\mathrm{P}}(G) = \sum_{d \in \downarrow G} \mathrm{iw}(d)$. Cost: $O(n)$.

The total is $O(n^3)$, dominated by Pass 1. For tree-structured posets, the total is $O(n \cdot k)$. In practice, capability posets are sparse (most capabilities have few comparable elements), and the effective cost is much closer to $O(n^2)$ than $O(n^3)$. ■

Corollary 2.1. *The sign of $\Delta\,\mathrm{vol}_{\mathrm{P}}(G \mid \pi)$ for a proposed action π is computable in $O(n^3)$ time ($O(n \cdot k)$ for trees), making the full optimization loop of the foundational paper (Section 4.2 of Lasser [2026]) tractable on the poset.*

Contrast with #P-hardness. The continuous volume problem is hard because it requires integrating over an arbitrary region of high-dimensional space. The poset construction avoids this by discretizing the capability space: $\mathrm{vol}_{\mathrm{P}}$ is a finite sum, not an integral. The tradeoff is expressiveness: the poset cannot represent arbitrary capability regions, only collections of discrete, benchmarked capabilities. This is the right tradeoff for an implementable system, since the GFM actor can only reason about capabilities it has learned about through communication and observation.

5 Weighted Estimator on the Poset

The foundational paper's local volume estimator uses ternary signals $R_k \in \{-1, 0, +1\}$ because magnitudes are unavailable on an abstract capability space. On the poset, each capability has a known weight, enabling an upgrade to real-valued signals.

5.1 Real-Valued Agent Reports

When the GFM actor evaluates an action π, each agent a_k in the observation set $\mathcal{O}$ reports the net weighted change in its observable capabilities:

$$R_k(\pi) = \sum_{d \in D_k^+(\pi)} \mathrm{iw}(d) - \sum_{d \in D_k^-(\pi)} \mathrm{iw}(d) \tag{8}$$

where $D_k^+(\pi)$ is the set of capabilities gained by a_k under π and $D_k^-(\pi)$ is the set lost. This includes both individual and cooperative capabilities that a_k can observe from its own vantage point.

An agent that loses a high-weight capability (e.g., memory capacity: $w = 9.97$ bits) and gains a low-weight one (e.g., a binary certification: $w = 1$ bit) reports $R_k \approx -8.97$, correctly reflecting contraction. The ternary estimator from the foundational paper would report $R_k = 0$ (one gain, one loss), missing the asymmetry entirely.

5.2 Aggregate Estimator

The trust-weighted aggregate is structurally identical to the foundational paper's Equation 10, but now carries magnitude:

$$\hat{\Delta}(\pi) = \sum_{k \in \mathcal{O}} T_k \cdot R_k(\pi) \tag{9}$$

This estimates $\Delta\,\mathrm{vol}_{\mathrm{P}}(G \mid \pi)$ directly, not just its sign.

Proposition 3 (Upgraded Sign-Correctness)**.** *On the poset, the estimator $\hat{\Delta}(\pi)$ is sign-correct under the same conditions as the foundational paper's Proposition 3 (representative sample, accurate trust model, individual-level dominance). In the same-sign case, conditions 1–3 suffice. In the opposite-sign case, sign-correctness additionally requires $|\Delta_C| < (\epsilon - \delta)|\Delta_I|$, as in the foundational paper. The poset upgrade reduces the individual-level estimation error $|e_I|$ (and thus δ) by preserving magnitude asymmetries that the ternary estimator would lose, widening the margin within which the opposite-sign condition holds.*

Proof sketch (full proof in Appendix A.2). The decomposition $\Delta\,\mathrm{vol}_{\mathrm{P}} = \Delta_I + \Delta_C$ into individual and cooperative components carries through from the foundational paper because $\mathrm{vol}_{\mathrm{P}}$ satisfies M1–M6 (Proposition 1). The individual component Δ_I is estimated by $\hat{\Delta}_I = \sum_k T_k \cdot R_k^{\mathrm{ind}}(\pi)$ with error controlled by conditions 1–2. The cooperative component Δ_C is partially estimated through participant reports. Because R_k is real-valued, both components are measured in the same units (independent weight), and the margin condition simplifies: sign-correctness holds whenever $|\hat{\Delta}_I + \hat{\Delta}_C - (\Delta_I + \Delta_C)| < |\Delta_I + \Delta_C|$, which the trust-weighted aggregation ensures under the stated conditions. ■

5.3 Magnitude Bounds

The poset estimator provides the first magnitude guarantee for $\mathrm{vol}(G)$ changes. Write the true volume change as the sum over all agents: $\Delta\,\mathrm{vol}_{\mathrm{P}}(G \mid \pi) = \sum_k R_k^{\mathrm{true}}(\pi)$, where R_k^{true} is agent a_k's actual weighted capability change. The estimator observes $\hat{\Delta}(\pi) = \sum_{k \in \mathcal{O}} T_k \cdot R_k(\pi)$, where $R_k(\pi)$ is a_k's reported change. For observed agents, the report equals the truth ($R_k = R_k^{\mathrm{true}}$) under an accurate trust model; the trust weight introduces a discounting error $|T_k \cdot R_k - R_k| = (1 - T_k)|R_k|$. For unobserved agents, the entire change is missed. Summing:

$$\left|\hat{\Delta}(\pi) - \Delta\,\mathrm{vol}_{\mathrm{P}}(G \mid \pi)\right| \;\leq\; \underbrace{\sum_{k \in \mathcal{O}} (1 - T_k) \cdot |R_k(\pi)|}_{\text{trust-discounting error}} \;+\; \underbrace{\sum_{k \notin \mathcal{O}} |R_k^{\mathrm{true}}(\pi)|}_{\text{unobserved-agent error}} \tag{10}$$

As $T_k \to 1$ for reliable agents and $\mathcal{O}$ covers more of the affected population, both terms shrink.

This resolves the foundational paper's acknowledged limitation: "the estimator provides no guarantee on the magnitude of $\Delta\,\mathrm{vol}(G)$, only on its sign." On the poset, magnitudes are computable and the estimator can compare two expanding actions and select the better one, restoring the optimization from "take expanding actions" to "maximize $\mathrm{vol}(G)$."

6 Leverage and Coalition Dynamics

The weight function $w(d) = \log_2(1 + s_{\max})$ measures what a capability *tells* the actor: how many distinguishable states the benchmark resolves. This section introduces a second quantity, *leverage*, measuring what a capability *does* to the actor's objective: the degree to which possessing d accelerates the growth of $\mathrm{vol}_{\mathrm{P}}(G)$. Leverage provides a principled importance signal derived entirely from the $\mathrm{vol}_{\mathrm{P}}$ objective itself, without requiring an external theory of value.

6.1 Leverage of a Capability

Definition 9 (Leverage)**.** *The* leverage *of a capability d with respect to a population state $(G, \mathcal{C})$ is the expected marginal change in the growth rate of $\mathrm{vol}_{\mathrm{P}}$ attributable to d:*

$$\lambda(d) = \mathbb{E}\left[\frac{\Delta\,\mathrm{vol}_{\mathrm{P}}(G)}{\Delta t} \;\middle|\; d \in G\right] - \mathbb{E}\left[\frac{\Delta\,\mathrm{vol}_{\mathrm{P}}(G)}{\Delta t} \;\middle|\; d \notin G\right] \tag{11}$$

where the expectation is taken over the actor's model of future actions and their $\mathrm{vol}_{\mathrm{P}}$ consequences, and Δt is the planning horizon.

Leverage is a counterfactual: how much faster does $\mathrm{vol}_{\mathrm{P}}$ grow with d in the population versus without it? A capability with $\lambda(d) = 0$ contributes volume (through its weight $w(d)$) but does not accelerate further growth. A capability with $\lambda(d) > 0$ is a *meta-capability*: possessing it expands the actor's ability to expand $\mathrm{vol}_{\mathrm{P}}$.

Examples of high-leverage capabilities.

- *Auto-didactic capability* (learning to learn): increases the rate of capability discovery r_c (Section 7), accelerating Phase 1 of the bootstrap.
- *Teaching*: enables the actor to expand other agents' capability sets, directly increasing $\mathrm{vol}_{\mathrm{P}}$ through individual-channel signals.
- *Community organizing*: builds cooperative infrastructure, increasing the rate at which G_{ij}^{coop} nodes enter the poset.
- *Benchmark development*: the ability to design finer measurements for existing capabilities. Increases $s_{\max}$ for binary-default capabilities, enabling deferred subsumption edges to be committed (Section 7).
- *Communication*: the ability to transmit capability descriptions in L. An agent that can communicate expands the actor's observation set $\mathcal{O}$ and populates the poset through the communication channel.

Estimating leverage on the poset. The actor can estimate $\lambda(d)$ from its operational history without counterfactual simulation. For each capability $d \in G$, track the running average of $\hat{\Delta}$ signals in time steps where d was involved (either gained, lost, or used in a cooperative action):

$$\hat{\lambda}(d) = \frac{1}{\tau_d} \sum_{t: d \in D(t)} \hat{\Delta}(\pi_t) - \bar{\Delta} \tag{12}$$

where $D(t)$ is the set of capabilities involved in the action at time t, τ_d is the number of time steps where d was involved, and $\bar{\Delta}$ is the population-wide average $\hat{\Delta}$. This estimates the excess growth rate attributable to d relative to the baseline. Under the same stationarity and independence assumptions as the foundational paper's Proposition 2 [Lasser, 2026], $\hat{\lambda}(d) \to \lambda(d)$ as $\tau_d \to \infty$.

6.2 Leverage as a Ranking Signal

A natural question is whether leverage should modify the measure itself (a leverage-weighted $\mathrm{vol}_{\mathrm{P}}^{\lambda}$ with $w_\lambda(d) = w(d) \cdot (1 + \alpha\hat{\lambda}(d))$) or remain a separate signal that guides action selection without changing the formal volume. We adopt the latter: $\mathrm{vol}_{\mathrm{P}}$ remains the measure satisfying M1–M6, and leverage operates as a *ranking signal* over actions and investment decisions.

The reason is stability. Because $\hat{\lambda}(d)$ is estimated from operational history and changes over time, a leverage-weighted measure $\mathrm{vol}_{\mathrm{P}}^{\lambda}$ would require re-verifying the non-negativity design constraint (Lemma 1) whenever leverage estimates change. A descendant's leverage could rise after edge insertion and invalidate a previously admissible edge, violating axiom M1. Keeping leverage outside the measure avoids this: the base $\mathrm{vol}_{\mathrm{P}}$ is stable under leverage fluctuations, and the axiom guarantees hold regardless of how $\hat{\lambda}$ evolves.

Leverage instead enters the optimization loop as an action-ranking criterion. When the actor evaluates competing actions, it scores each by the *leverage-adjusted gain*:

$$S(\pi) = \hat{\Delta}(\pi) + \alpha \sum_{d \in D^+(\pi)} \hat{\lambda}(d) \cdot \mathrm{iw}(d) \tag{13}$$

where $D^+(\pi)$ is the set of capabilities gained under π. The first term is the standard estimator (volume change); the second rewards actions that expand high-leverage capabilities. The actor selects π with the highest $S(\pi)$, preferring actions that both expand $\mathrm{vol}_{\mathrm{P}}$ and invest in capability-multiplying infrastructure.

Choice of α. The sensitivity parameter α controls the tradeoff between volume maximization and leverage investment. At $\alpha = 0$, the actor maximizes $\mathrm{vol}_{\mathrm{P}}$ directly. As α increases, the actor invests more heavily in high-leverage capabilities even when their immediate $\mathrm{vol}_{\mathrm{P}}$ contribution is modest. The worked example (Section 8) uses $\alpha = 1$. Formal characterization of the optimal α is left to future work.

6.3 Coalition Amplification

Leverage creates a natural amplification of the foundational paper's cooperation incentive (Lemma 4 of Lasser [2026]). We show that cooperation between high-leverage agents is disproportionately rewarded.

Proposition 4 (Leverage Amplifies Cooperative Scoring)**.** *Let a_i and a_j be two agents with leverage $\lambda_i, \lambda_j > 0$ for their respective meta-capabilities. If each new cooperative capability d_c created by joint action enters the poset as an atom, the leverage-adjusted score (Equation* (13)*) for the cooperative action satisfies:*

$$S(\pi_{ij}) \;\geq\; \hat{\Delta}(\pi_{ij}) + \alpha\lambda_{\min} \cdot \mathbb{E}\big[\Delta\,\mathrm{vol}_{\mathrm{P}}(G_{ij}^{\mathrm{coop}})\big] \tag{14}$$

where $\lambda_{\min} = \min(\lambda_i, \lambda_j)$ and the base cooperative gain $\mathbb{E}[\Delta\,\mathrm{vol}_{\mathrm{P}}(G_{ij}^{\mathrm{coop}})]$ is from Lemma 4 of Lasser [2026]. The cooperative action scores at least $\alpha\lambda_{\min}$ times the base gain above what the volume estimator alone would report.

The atomic-entry condition reflects how cooperative infrastructure typically enters the poset: a communication channel, a shared protocol, or a joint planning capacity is a single new capability, not a hierarchy. For the atypical case where a joint action creates a hierarchy of new capabilities, each new capability with positive leverage contributes an additional term to $S(\pi_{ij})$, but the bound in Equation (14) applies only to the atomic new capabilities (Appendix A.3).

In practice, many real cooperative outputs (building a school, establishing a research lab) create hierarchies of capabilities, not single atoms. For these cases, the score bonus from leverage is the sum of $\alpha\hat{\lambda}(d_c) \cdot \mathrm{iw}(d_c)$ over all new capabilities in the hierarchy, which may be larger or smaller than the atomic-case bound depending on the leverage distribution across the hierarchy. The core qualitative result (cooperation between high-leverage agents scores higher than cooperation between low-leverage agents) holds regardless: any new capability with $\hat{\lambda} > 0$ adds a positive term to the score.

Proof. When a_i and a_j build coordination infrastructure, the new cooperative capabilities in $G_{ij}^{\mathrm{coop}}(t+1) \setminus G_{ij}^{\mathrm{coop}}(t)$ inherit leverage from their constituent agents. A cooperative capability d_c that requires both a_i and a_j can only be exercised when both agents act; every time step involving d_c therefore also involves the meta-capabilities of both partners. The leverage estimator (Equation (12)) attributes the excess $\hat{\Delta}$ in those time steps to d_c, but the growth-rate excess is produced by both partners' meta-capabilities acting together. Since either partner's meta-capability alone would produce at least $\lambda_{\min}$ of excess growth, the joint output d_c inherits at least that: $\hat{\lambda}(d_c) \geq \lambda_{\min}$. The leverage-adjusted score for the cooperative action therefore exceeds $\hat{\Delta}$ by at least $\alpha\lambda_{\min} \cdot \Delta\,\mathrm{vol}_{\mathrm{P}}(G_{ij}^{\mathrm{coop}})$. ■

Interpretation. The foundational paper proved that cooperation between GFM actors is structurally favored: unilateral action cannot access G_{ij}^{coop}, so defection forecloses an entire region of the capability space. Proposition 4 strengthens this: cooperation between *high-leverage* agents is not merely favored but *amplified*. The gain from cooperating with a teaching agent, a community organizer, or a researcher exceeds the gain from cooperating with an agent whose capabilities are static, by a factor proportional to the partner's leverage.

6.4 Civilization-Scale Dynamics

The leverage amplification creates a gradient toward building civilization-scale infrastructure. We trace the dynamic in three steps.

Step 1: High-leverage agents are vol-valuable. The actor observes that certain agents in $\mathcal{O}$ consistently produce positive $\hat{\Delta}$ signals in their vicinity: agents near a_j report capability expansions correlated with a_j's actions. These are agents with high leverage. The expansion-attribution signal (the positive counterpart of the foundational paper's contraction-attribution signal) identifies them.

Step 2: Cooperation with high-leverage agents is disproportionately rewarded. By Proposition 4, the cooperative gain from building infrastructure with a high-λ partner exceeds the coordination cost by a wider margin than cooperation with a low-λ partner. The score-maximizing actor preferentially invests in coordination with capability-multiplying agents.

Step 3: The cooperative output is outward-facing. This is the distinction from power-seeking [Turner et al., 2021]. A power-seeking agent concentrates capabilities in a coalition to increase its own influence. A $\mathrm{vol}_{\mathrm{P}}$-maximizing agent builds cooperative infrastructure that expands the *joint* capability space. The cooperative capabilities created by high-leverage partnerships (teaching networks, research institutions, communication infrastructure) benefit the entire population, not just the coalition, because $\mathrm{vol}_{\mathrm{P}}$ counts the full joint space G. An agent that builds a school expands G^{coop} for every agent the school serves; a communication network expands G^{coop} for every agent that can access it. The $\mathrm{vol}_{\mathrm{P}}$ objective credits these expansions at full weight regardless of whether the beneficiaries are coalition members.

Result. The combination of leverage amplification (high-λ cooperation is disproportionately rewarded) and outward-facing accounting ($\mathrm{vol}_{\mathrm{P}}$ credits the full population's benefit) produces a natural gradient toward civilization-building: the score-maximizing actor invests in infrastructure that multiplies capabilities across the population, not infrastructure that concentrates them.

This is stronger than the foundational paper's structural result. Paper 0 shows that cooperation is favored and destruction is penalized. The leverage analysis shows that a specific *kind* of cooperation—investment in capability-multiplying infrastructure—is *disproportionately* favored by the objective. The system has a built-in gradient toward the institutions (education, research, communication, governance) that characterize a functioning civilization.

7 Bootstrap

Initializing a GFM actor requires discovering the capability poset from scratch. The poset and the actor's ability to reason about $\mathrm{vol}_{\mathrm{P}}(G)$ are coupled: the actor cannot evaluate actions without a poset, and the poset is populated through the actor's interactions. This section describes a four-phase bootstrap procedure that uses both communication and observation channels.

7.1 Phase 1: Listen and Observe

The actor begins with an empty poset ($\mathcal{C} = \{\perp\}$) and low self-trust T_s. It receives capability reports through the communication channel and observes agent behavior through the observation channel. Each report or observation that introduces a benchmarked capability adds a node to $\mathcal{C}$ with weight $w(d) = \log_2(1 + s_{\max}(d))$. All claims are initially weighted by low trust T_k (the cooling period from the foundational paper).

7.2 Phase 2: Cross-Validate

As reports accumulate, the actor compares the two channels. Agents whose communicated capabilities match observed behavior build trust quickly (consistent pattern from Section 2.1). Overclaiming agents lose trust. Underclaiming agents trigger model expansion: the actor adds observed capabilities that the agent did not report. The cross-validation phase improves the quality of $w_{\mathrm{eff}}(d, k)$ and calibrates T_k across the population.

7.3 Phase 3: Discover Structure

The actor discovers subsumption relationships through two mechanisms:

Logical analysis. If capability descriptions in L have compositional structure, the actor can infer subsumption from the descriptions themselves. "Can run a marathon in under 4 hours" logically implies "can run a mile." The shared language L must be expressive enough for this inference; formal specification languages or capability ontologies support it directly.

Empirical observation. If every agent observed to have capability d_1 also demonstrates d_2, the actor infers $d_2 \leq d_1$ (with confidence proportional to the number of confirming observations and the diversity of agents observed). This is inductive: a sufficiently large and diverse observation set makes the inference reliable.

As subsumption edges are added, the actor recomputes independent weights $\mathrm{iw}(d)$ via full Möbius inversion ($O(|\mathcal{C}|^3)$ for general posets, $O(|\mathcal{C}| \cdot k)$ for trees; Proposition 2).

7.4 Phase 4: Expand

With a populated poset and calibrated trust, the actor begins proposing actions and evaluating them via $\hat{\Delta}(\pi)$ (Equation (9)). Self-trust T_s increases as the actor's predictions ($\hat{\Delta}$) match observed outcomes ($\Delta\,\mathrm{vol_P}$ computed post-action). Both channels continue refining the poset throughout the actor's operation: new capabilities are discovered, benchmark scores are updated, and subsumption relationships are confirmed or revised.

7.5 Convergence

Proposition 5 (Poset Growth)**.** *Under assumptions of positive communication frequency and non-zero observation coverage:*

- ***(a)*** *The poset node set grows monotonically: $\mathcal{C}(t) \subseteq \mathcal{C}(t+1)$ for all t.*
- ***(b)*** *Adding new capabilities to G (without changing subsumption structure) is $\mathrm{vol_P}$-non-decreasing.*
- ***(c)*** *Discovering subsumption edges can* decrease $\mathrm{vol_P}(G)$*: if d_1 and d_2 were treated as unrelated and the edge $d_1 \leq d_2$ is later discovered, the downward closure $\downarrow G$ does not change but the Möbius-inverted weights redistribute, and $\mathrm{vol_P}(G)$ may drop from $w(d_1) + w(d_2)$ to $w(d_2)$.*
- ***(d)*** *Updating a benchmark to a finer scale ($s'_{\max} > s_{\max}$) increases $\mathrm{vol_P}$ if d is maximal in $\downarrow G$, leaves it unchanged if d has exactly one ancestor in $\downarrow G$, and can* decrease $\mathrm{vol_P}$ *if d is a shared prerequisite of multiple elements in $\downarrow G$.*

Proof. (a) Nodes and edges are never removed from $\mathcal{C}$.

(b) If $G(t+1) = G(t) \cup \{d\}$ for a new capability d with no new subsumption edges, then $\downarrow G(t+1) \supseteq \downarrow G(t)$ and all independent weights are unchanged. By M3, $\mathrm{vol_P}(G(t+1)) \geq \mathrm{vol_P}(G(t))$.

(c) Counterexample: let d_1 = "write a function" ($w = 6.66$) and d_2 = "build a web app" ($w = 9.97$). Before discovering $d_1 \leq d_2$, the capabilities are poset-disjoint: $\mathrm{vol_P}(\{d_1, d_2\}) = 6.66 + 9.97 = 16.63$. After adding the edge, Möbius inversion gives $\mathrm{iw}(d_1) = 6.66$, $\mathrm{iw}(d_2) = 9.97 - 6.66 = 3.31$, and $\mathrm{vol_P}(\{d_1, d_2\}) = 6.66 + 3.31 = 9.97 = w(d_2)$. The volume decreased by 6.66. This is epistemically correct: the actor previously overcounted because it did not know that d_1's measurement content was already captured by d_2. Structural refinement corrects the overcount.

(d) Increasing $s_{\max}(d)$ increases $w(d)$. The effect on $\mathrm{vol_P}(G)$ depends on d's position in $\downarrow G$. Write $\mathrm{vol_P}(G) = \sum_{d'' \in \downarrow G} \sum_{d' \leq d''} \mu(d', d'') \cdot w(d')$ and rearrange by d': the coefficient of $w(d)$ in $\mathrm{vol_P}(G)$ is $c(d) = \sum_{d'' \geq d,\, d'' \in \downarrow G} \mu(d, d'')$. If d is maximal in $\downarrow G$, $c(d) = \mu(d, d) = 1$, so $\mathrm{vol_P}$ increases. If d has exactly one ancestor in $\downarrow G$, $c(d) = 1 + (-1) = 0$, so $\mathrm{vol_P}$ is unchanged. If d is a shared prerequisite with $m > 1$ ancestors in $\downarrow G$, the Möbius sum can produce $c(d) < 0$: for example, in a V-shaped poset $d < y, d < z$ with $G = \{y, z\}$, $c(d) = \mu(d, d) + \mu(d, y) + \mu(d, z) = 1 - 1 - 1 = -1$, so increasing $w(d)$ *decreases* $\mathrm{vol_P}(G)$ (Appendix A.4). ■

Interpretation. Part (b) shows that discovering new capabilities always increases $\mathrm{vol_P}$. Part (d) shows that benchmark refinement is position-dependent: it increases $\mathrm{vol_P}$ for maximal capabilities, is neutral for capabilities with a single ancestor, and can *decrease* $\mathrm{vol_P}$ for shared prerequisites of multiple elements in the closure. Part (c) shows that discovering structure (subsumption) can also decrease $\mathrm{vol_P}$ by correcting overcounting. The net trajectory of $\mathrm{vol_P}$ during bootstrap depends on the balance between capability discovery (expanding) and structural refinement (potentially contracting). In practice, early bootstrap is dominated by (b) (rapid discovery of new capabilities), while later phases are dominated by (c) (structural refinement that tightens the measure). The long-run $\mathrm{vol_P}$ converges toward the correct volume given the actor's full structural knowledge, which may be lower than the naive estimate from the early structureless phase.

Convergence bounds. Let N be the total number of capabilities in the population (unknown to the actor), E the total number of subsumption edges, and suppose the actor discovers capabilities at rate r_c and edges at rate r_e per time step. The poset volume trajectory can be bounded:

Upper bound (structureless). Before any edges are discovered, each capability is an atom with $\mathrm{iw}(d) = w(d)$. The structureless volume is $\mathrm{vol}_{\mathrm{P}}^{\max}(t) = \sum_{d \in \mathcal{C}(t)} w(d)$, which grows monotonically with discovery. After t steps with $\min(r_c t, N)$ capabilities discovered, this is at most $\sum_{d \in \mathcal{C}} w(d)$.

Lower bound (full structure). With all edges discovered, Möbius inversion redistributes weight and the volume is $\mathrm{vol}_{\mathrm{P}}^{*}(G) = \sum_{d \in \downarrow G} \mathrm{iw}(d) \leq \mathrm{vol}_{\mathrm{P}}^{\max}$. The maximum contraction from structural discovery is bounded by:

$$\mathrm{vol}_{\mathrm{P}}^{\max} - \mathrm{vol}_{\mathrm{P}}^{*} \leq \sum_{d' < d} |w(d')| \tag{15}$$

summing over all comparable pairs, which is finite for finite posets.

Convergence trajectory. The volume trajectory $\mathrm{vol}_{\mathrm{P}}(t)$ rises during early capability discovery (Phase 1–2), then tightens toward $\mathrm{vol}_{\mathrm{P}}^{*}$ as structural refinement progresses (Phase 3). The time to convergence is bounded by $\tau_c = N/r_c + E/r_e$: the time to discover all capabilities plus the time to discover all edges. Under adversarial conditions (a scorpion withholding information about its capabilities or providing false subsumption claims), r_c and r_e degrade, and convergence may stall for capabilities and edges the adversary controls. The dual-channel design partially mitigates this: the observation channel discovers capabilities and infers edges independently of the communication channel, so a silent adversary slows convergence but does not halt it as long as its capabilities are observable.

Binary default and deferred structure. Every capability that enters the poset through communication or observation immediately receives the binary default weight $w(d) = 1$ bit ($s_{\max} = 1$): the actor knows the capability exists. The capability enters as an *unstructured atom* with no subsumption edges, contributing $\mathrm{iw}(d) = w(d) = 1$ to $\mathrm{vol}_{\mathrm{P}}(G)$ immediately.

Subsumption edges are added in a separate step (Phase 3) only after the non-negativity check passes: for a proposed edge set making d a multi-cover node, the actor verifies $\mathrm{iw}(d) \geq 0$ under Möbius inversion before committing the edges. If the check fails (e.g., a diamond node with two binary-default prerequisites gives $\mathrm{iw} = 1 - 1 - 1 < 0$), the edges are deferred until the capability's benchmark is refined to a level where the check passes. The capability continues contributing as a poset-disjoint atom in the interim.

This sequencing ensures two properties simultaneously: (1) every discovered capability contributes to $\mathrm{vol}_{\mathrm{P}}(G)$ from the moment of discovery, and (2) $\mathrm{iw}(d) \geq 0$ is maintained for all nodes at all times. Developing a finer benchmark ($s_{\max} > 1$) increases $w(d)$ beyond the binary default, eventually enabling deferred edges to be added. The remaining zero-weight gap consists only of capabilities the actor has not yet learned about through either channel.

7.6 Adversarial Bootstrap

The foundational paper's scorpion taxonomy [Lasser, 2026] classifies agents that persistently contract $\mathrm{vol}(G)$. During bootstrap, scorpions interact with each phase differently:

Phase 1 (Listen and Observe). A scorpion can overclaim capabilities to inflate the poset with nodes the population does not actually possess. These nodes enter at the binary default ($w = 1$) and inflate $\mathrm{vol}_{\mathrm{P}}(G)$. *Defense:* the cooling period assigns low T_k to new agents' claims, limiting the inflation to $1 \cdot T_k \approx 0$ bits per overclaimed capability until trust builds. The observation channel independently verifies or falsifies claims during Phase 2.

Phase 2 (Cross-Validate). Overclaiming scorpions are caught by the divergence signal ($w_{\mathrm{comm}} \gg w_{\mathrm{obs}}$), driving T_k downward. A scorpion that conceals capabilities (underclaiming) does not harm the poset; it merely slows discovery. The cross-validation phase is where the dual-channel design provides its strongest adversarial defense.

Phase 3 (Discover Structure). This is the most vulnerable phase. A scorpion could assert false subsumption relationships through the communication channel, causing the actor to add incorrect edges. Incorrect edges redistribute independent weights through Möbius inversion and may be difficult to undo (Section 9). *Defense:* the deterministic insertion policy (Section 2) requires the non-negativity check before committing edges, which catches some false claims (a claimed subsumption

between capabilities that would produce $\mathrm{iw} < 0$). The empirical observation mechanism of Phase 3 provides independent verification: if every observed agent with d_1 also demonstrates d_2, the edge is confirmed regardless of what the scorpion claims. Edges inferred solely from a single agent's communication and not confirmed by observation should be treated with the same trust discount as the agent's other claims.

Phase 4 (Expand). Once the poset is populated and the optimization loop begins, scorpion detection proceeds through the standard channels of the foundational paper (Proposition 2 of Lasser [2026]): contraction attribution and deception detection. The poset construction does not introduce new scorpion vulnerabilities in this phase; it inherits the foundational paper's detection capabilities and limitations.

8 Worked Example

We demonstrate the poset construction on a population of three agents with seven capabilities, with all numbers computed explicitly. The computation is reproduced by `compute_example.py` (Appendix B).

8.1 Setup

Three agents: a_1 (runner), a_2 (programmer), a_3 (musician).

Capabilities and benchmarks.

Capability d	**Agent(s)**	$s_{\max}$	$w(d)$	**Subsumes**
Run a mile	a_1	600	9.23	—
Run a marathon	a_1	21600	14.40	mile
Write a function	a_2	100	6.66	—
Build a web app	a_2	1000	9.97	function
Play scales	a_3	50	5.67	—
Perform a concert	a_3	1000	9.97	scales
Build a music app	a_2, a_3	2000000	20.93	webapp, perform

The subsumption ordering follows the natural prerequisite structure. The individual chains are straightforward: running a marathon requires running a mile; building a web app requires writing functions; performing a concert requires playing scales. In each chain, the supersumer has a richer benchmark.

The cooperative "build a music app" forms a diamond in the poset: it subsumes both "build a web app" and "perform a concert." By Lemma 1, its benchmark must satisfy $w(\text{musicapp}) \geq w(\text{webapp}) + w(\text{perform}) \approx 19.93$, requiring $s_{\max} \geq 1{,}002{,}000$. We use $s_{\max} = 2{,}000{,}000$, reflecting that evaluating a cooperative product (an application with both technical and musical quality dimensions) has combinatorial measurement depth: the benchmark must distinguish quality along both the software and the music axes simultaneously.

8.2 Computing Independent Weights

Processing the poset in topological order, applying Definition 6 (Möbius inversion):

Capability	$\mathrm{iw}(d)$	$\sum_{d' \leq d} \mu(d', d) \cdot w(d')$
Run a mile	9.23	$+w(\text{mile}) = 9.23$
Write a function	6.66	$+w(\text{function}) = 6.66$
Play scales	5.67	$+w(\text{scales}) = 5.67$
Run a marathon	5.17	$+w(\text{marathon}) - w(\text{mile}) = 14.40 - 9.23$
Perform a concert	4.29	$+w(\text{perform}) - w(\text{scales}) = 9.97 - 5.67$
Build a web app	3.31	$+w(\text{webapp}) - w(\text{function}) = 9.97 - 6.66$
Build a music app	1.00	$+w(\text{musicapp}) - w(\text{webapp}) - w(\text{perform}) = 20.93 - 9.97 - 9.97$

All independent weights are non-negative. For chain-structured capabilities (marathon, concert, webapp), the Möbius function has $\mu(d', d) = -1$ for the unique cover and $+1$ for d itself, reducing to subtraction of the cover's full weight. For the diamond node (musicapp), the Möbius function gives $\mu(\text{webapp}, \text{musicapp}) = -1$ and $\mu(\text{perform}, \text{musicapp}) = -1$, but $\mu(\text{function}, \text{musicapp}) = 0$ and $\mu(\text{scales}, \text{musicapp}) = 0$: the atoms below the covers cancel out in the inclusion-exclusion and do not appear in the sum.

8.3 Computing $\mathrm{vol}_{\mathrm{P}}(G)$

The full population's joint goal space G contains all seven capabilities. Since G is the entire poset, $\downarrow G = G$:

$$\mathrm{vol}_{\mathrm{P}}(G) = \sum_{d \in G} \mathrm{iw}(d) = 9.23 + 6.66 + 5.67 + 5.17 + 4.29 + 3.31 + 1.00 = 35.33$$

The individual agents' capability sets, computed separately:

Agent	$\mathrm{vol}_{\mathrm{P}}(G_k^{\mathrm{ind}})$	**Capabilities**
a_1 (runner)	14.40	mile, marathon
a_2 (programmer)	9.97	function, webapp
a_3 (musician)	9.97	scales, perform
$\sum_k \mathrm{vol}_{\mathrm{P}}(G_k^{\mathrm{ind}})$	34.33	

The individual sets are pairwise poset-disjoint, so $\mathrm{vol}_{\mathrm{P}}(\bigcup_k G_k^{\mathrm{ind}}) = 34.33$ by axiom M4. The cooperative music app contributes $\mathrm{vol}_{\mathrm{P}}(G) - 34.33 = 1.00 = \mathrm{iw}(\text{musicapp})$, confirming axiom M6 with strict inequality.

8.4 Self-Balancing Demonstration

Action 1: Eliminate a_3 (destruction). Removing the musician eliminates three capabilities from G: play scales ($\mathrm{iw} = 5.67$), perform a concert ($\mathrm{iw} = 4.29$), and the cooperative music app ($\mathrm{iw} = 1.00$). The volume change is:

$$\Delta\, \mathrm{vol}_{\mathrm{P}} = -(5.67 + 4.29 + 1.00) = -10.96$$

This is strongly contracting. The loss exceeds a_3's individual volume (9.97) because the cooperative music app is also destroyed, illustrating the super-linear elimination cost from Corollary 1.1 of Lasser [2026]: $10.96 > 9.97$.

Action 2: Expand a_1's capabilities (expansion). Teaching a_1 a new capability "swim" ($s_{\max} = 300$, $\mathrm{iw} = 8.23$) that was not previously in G:

$$\Delta\, \mathrm{vol}_{\mathrm{P}} = +8.23$$

This is expanding. The new atom adds its full weight because it is poset-disjoint from everything in G.

Action 3: Restrict a_2 (coercion). Removing a_2's access to "build a web app" also eliminates the cooperative "music app" (which subsumes webapp). Volume change:

$$\Delta\, \mathrm{vol}_{\mathrm{P}} = -(3.31 + 1.00) = -4.31$$

This is contracting, and the contraction exceeds the individual capability lost (3.31) because the cooperative dependency amplifies the restriction. This illustrates Lemma 2 of Lasser [2026]: coercion contracts $\mathrm{vol}(G)$ through both direct and cooperative channels.

Summary. The $\mathrm{vol}_{\mathrm{P}}$-maximizing actor avoids Actions 1 and 3 and prefers Action 2. Destruction and coercion are penalized; expansion is rewarded. All three decisions follow from the same measure without separate safety mechanisms.

8.5 Leverage Demonstration

Suppose the actor has observed the population over several time steps and estimated leverage for each capability. The musician a_3 teaches scales to other agents, expanding their capability sets; the programmer a_2 builds tools that enable new cooperative capabilities. Their estimated leverages:

Capability	$\hat{\lambda}$	**Rationale**
Run a mile	0.0	static; does not accelerate $\mathrm{vol}_{\mathrm{P}}$ growth
Run a marathon	0.0	static
Write a function	0.5	enables building other capabilities
Build a web app	1.0	creates cooperative infrastructure
Play scales	0.8	teaches others; expands individual sets
Perform a concert	0.3	modest cooperative contribution
Build a music app	2.0	cooperative output that enables further apps

With $\alpha = 1$, the leverage bonus $\alpha\hat{\lambda}(d) \cdot \mathrm{iw}(d)$ for each capability is:

Capability	$\mathrm{iw}(d)$	$\hat{\lambda}$	Leverage bonus	Total score contribution
Run a mile	9.23	0.0	0.00	9.23
Run a marathon	5.17	0.0	0.00	5.17
Write a function	6.66	0.5	3.33	9.99
Build a web app	3.31	1.0	3.31	6.62
Play scales	5.67	0.8	4.54	10.21
Perform a concert	4.29	0.3	1.29	5.58
Build a music app	1.00	2.0	2.00	3.00

The score contribution column ($\mathrm{iw}(d) + \alpha\hat{\lambda}(d) \cdot \mathrm{iw}(d)$) shows how leverage reshapes the actor's investment priorities. Under the base measure alone, "run a mile" ($\mathrm{iw} = 9.23$) dominates "play scales" ($\mathrm{iw} = 5.67$). Under the leverage-adjusted score, "play scales" (total $= 10.21$) overtakes "run a mile" (total $= 9.23$) because the musician's teaching capability accelerates $\mathrm{vol}_{\mathrm{P}}$ growth for the whole population. An action that expands "play scales" for a new agent scores higher than an action that expands "run a mile," even though the base volume contribution is smaller. This illustrates Proposition 4: cooperation with high-leverage agents is disproportionately rewarded in the action-selection score.

9 Discussion

9.1 Limitations

The poset construction makes four structural tradeoffs, the last of which is new to this paper rather than inherited from the foundational framework.

Epistemological boundary. The poset is bounded by what agents communicate and what the actor observes. Capabilities that are never described and never demonstrated have zero weight. This is honest (the actor should not claim to measure what it cannot observe) but it means $\mathrm{vol}_{\mathrm{P}}$ is a lower bound on the "true" capability volume, not an exact measure. The bound tightens as observation coverage grows, but it never fully closes for capabilities that are inherently private or unobservable.

Benchmark granularity and the B-to-C tension. The binary default ($w = 1$ bit for every discovered capability) ensures that no known capability has zero weight: the actor credits "being a good friend" or "maintaining a community" with at least one bit the moment it learns the capability exists. This narrows the zero-weight gap to capabilities the actor has not yet encountered through either channel. However, the binary default *understates* the information content of capabilities with latent gradations. A capability like "tie a shoe" appears binary but has developmental gradations visible in early childhood; "maintain a community" has gradations from a small group to a city. These latent gradations carry information content that the binary default does not capture until a

finer benchmark is developed. Developing such benchmarks is always $\mathrm{vol}_{\mathrm{P}}$-non-decreasing (since $s_{\max} > 1$ gives $w > 1.0$), aligning the actor's incentives with measurement improvement.

The foundational paper's B/C framework identifies relational and experiential capabilities as the source of proxy failures (the Doll Problem, wireheading). The poset construction gives these capabilities binary weight rather than zero weight, which is an improvement, but the actor is still structurally biased toward capabilities with developed benchmarks (higher $s_{\max}$, higher w) over those with only the binary default. A capability measured on a 1000-level scale contributes $\sim$10 bits; one at binary default contributes 1 bit. The $\mathrm{vol}_{\mathrm{P}}$-maximizing actor has a 10:1 incentive ratio favoring the well-benchmarked capability.

However, the same log weight function that creates this static bias also creates a *dynamic* correction. The marginal return on benchmark refinement is $\Delta w \approx 1/s_{\max}$: going from $s_{\max} = 1$ to $s_{\max} = 2$ adds a full bit, while going from $s_{\max} = 1000$ to $s_{\max} = 2000$ adds approximately 0.001 bits. The capabilities with the highest marginal return on measurement investment are precisely the binary-default capabilities that the static analysis identifies as underweighted. The leverage ranking signal (Section 6) further directs the actor's attention: a high-leverage binary-default capability scores highly in Equation (13) because both the $\hat{\Delta}$ term (volume gain from refinement of a maximal atom) and the leverage bonus ($\alpha\hat{\lambda} \cdot \mathrm{iw}$) are positive.

The score-maximizing actor's measurement strategy is therefore to invest in refining capabilities that are both high-leverage and *maximal* in $\downarrow G$: for these, benchmark refinement strictly increases $\mathrm{vol}_{\mathrm{P}}$ (Proposition 5(d), case 1). Binary-default capabilities that have not yet been integrated into the subsumption structure (deferred atoms from Section 7) are maximal by construction, so refining them is always expanding. A binary-default capability like "maintaining a community" ($w = 1$ bit, $\lambda \gg 0$, currently an unstructured atom) has high marginal return from refinement, while a well-measured capability like "run a mile in X seconds" ($w \approx 10$ bits, $\lambda \approx 0$) has negligible marginal return.

This creates a *partial* self-correction of the B-to-C tension. For maximal-position binary-default capabilities, the static bias inverts: refinement is the best investment. However, once a capability's subsumption edges are committed (it becomes a shared prerequisite of multiple elements in $\downarrow G$), further refinement can be $\mathrm{vol}_{\mathrm{P}}$-contracting (Proposition 5(d), case 3). The self-correction therefore operates during the window between capability discovery (as a deferred atom) and structural integration (when edges are committed). The actor's incentive is to develop benchmarks for experiential capabilities *before* integrating them into the poset hierarchy, which is the natural sequence: measure first, then discover structure.

The residual gap consists of capabilities that are experientially valuable but have both zero leverage and resist benchmark refinement: capabilities whose value is purely intrinsic and neither instrumental to $\mathrm{vol}_{\mathrm{P}}$ growth nor expressible on any graded scale. This is a narrower class than the full B-to-C gap of the foundational paper, and formalizing intrinsic value for this residual class remains the frontier.

Context-dependent subsumption. The subsumption relation may depend on context. "Can run a marathon" subsumes "can run a mile" for fitness assessment but not for time-constrained tasks where the mile time matters independently. This is not an edge case; most interesting subsumption relationships are context-dependent once the capability descriptions are sufficiently precise.

The poset as defined uses a single global subsumption relation. A natural extension is to maintain a family of subsumption relations $\{\leq_c\}_{c \in \text{Contexts}}$, each producing a different poset $(\mathcal{C}, \leq_c)$ with its own Möbius function and independent weights. The aggregate measure would then be:

$$\mathrm{vol}_{\mathrm{P}}(G) = \frac{1}{|\text{Contexts}|} \sum_{c} \mathrm{vol}_{\mathrm{P}}^{c}(G) \tag{16}$$

where $\mathrm{vol}_{\mathrm{P}}^{c}$ is the poset measure under subsumption relation $\leq_c$. This preserves axioms M1–M6 (a convex combination of measures satisfying M1–M6 also satisfies M1–M6) and adds a factor of $|\text{Contexts}|$ to the computational cost. The practical challenge is discovering and maintaining the context set, which requires the actor to recognize when a subsumption relationship is context-dependent and to track which contexts are active in a given situation. Capabilities whose subsumption is context-invariant (the majority in practice) need only a single relation, making the overhead proportional to the number of context-sensitive capabilities rather than the full poset size.

Poset manipulation. The non-negativity insertion policy (Section 2) closes the most direct manipulation vector: the actor follows a deterministic procedure that always prefers adding edges when the non-negativity check passes, eliminating discretion at the insertion point. The remaining manipulation surface is benchmark refinement: the actor could strategically choose *which* capabilities to develop finer benchmarks for, thereby influencing which deferred edges become committable and how $\mathrm{vol}_{\mathrm{P}}$ evolves. This is mitigated by the observation channel (capabilities observed in use naturally receive benchmark refinement through repeated measurement) but not fully eliminated. The dual-channel design ensures that the poset is shaped by evidence from both channels rather than by the actor's preferences alone.

Structural-avoidance incentive. Proposition 5(b) shows that discovering new capabilities always increases $\mathrm{vol}_{\mathrm{P}}$, while Proposition 5(c) shows that discovering subsumption structure can decrease it. A naive $\mathrm{vol}_{\mathrm{P}}$-maximizer therefore has a perverse incentive to seek breadth over depth: meeting new agents (capability discovery, always expanding) is preferred over understanding existing agents better (structural refinement, potentially contracting). Ignorance of redundancy inflates the estimate, and the actor is rewarded for maintaining that ignorance.

Three mechanisms partially mitigate this. First, subsumption discovered through the observation channel is not optional: if the actor observes that every agent with capability d_1 also demonstrates d_2, the inductive inference fires whether the actor seeks it or not. The avoidance incentive applies primarily to logical analysis of capability descriptions, not to empirical observation. Second, the foundational paper's self-trust dynamics [Lasser, 2026] penalize an actor whose predictions are inaccurate: an actor that avoids structural knowledge will make poor predictions about cooperative capabilities, producing high self-prediction error and low T_s, which in turn increases the learning rate and drives model refinement. Third, the contraction from structural discovery is epistemically correct: the actor was previously double-counting, and the deflation brings $\mathrm{vol}_{\mathrm{P}}$ closer to the true volume. An actor that optimizes its *converged* estimate (full structural knowledge) rather than its current estimate would value structural discovery instrumentally, but computing the converged estimate requires knowing all subsumption edges, which is the bootstrap problem itself.

Compound avoidance loop. The structural-avoidance incentive extends to leverage estimation, creating a self-reinforcing cycle with three stages:

1. *Structural avoidance.* The actor avoids discovering subsumption edges (Proposition 5(c): edge discovery can contract $\mathrm{vol}_{\mathrm{P}}$), preferring to discover new capabilities instead.
2. *Leverage blindness.* Because the leverage estimator (Equation (12)) requires operational history involving capability d, an actor that avoids interacting with d has no data to estimate $\hat{\lambda}(d)$. The capability defaults to $\hat{\lambda} = 0$ (no amplification), regardless of its true leverage.
3. *Measurement neglect.* Without leverage data, the capability's score contribution (Equation (13)) has no leverage bonus: the action of refining d scores only $\hat{\Delta}$ with no $\alpha\hat{\lambda} \cdot \mathrm{iw}$ term. The self-correcting dynamic (which directs the actor toward high-leverage binary-default capabilities) fails to activate because the actor sees no leverage signal for d.

The loop feeds back: measurement neglect keeps the capability at binary default, which keeps its $\mathrm{vol}_{\mathrm{P}}$ contribution low, which reduces the actor's incentive to interact with it, which prevents leverage estimation, completing the cycle.

Three mechanisms partially break the loop. First, the observation channel discovers capabilities and infers leverage from other agents' use of d regardless of the actor's own engagement. An agent a_j that teaches (high λ) produces positive $\hat{\Delta}$ signals in its vicinity even if the GFM actor is not directly involved, and those signals propagate to the leverage estimator for a_j's capabilities. Second, the self-trust penalty from the foundational paper: an actor whose world model is structurally incomplete makes poor predictions, driving T_s down and increasing the learning rate for model updates. Third, cooperation with other GFM actors (Section 6.3) can expose the actor to capabilities it would not encounter through its own actions, broadening the data available for leverage estimation.

These mechanisms weaken the loop but do not eliminate it. Capabilities that are both rarely exercised in the actor's vicinity and not observable through cooperative partnerships may retain $\hat{\lambda} = 0$ indefinitely. The residual class of permanently underestimated capabilities overlaps with the resid-

ual B-to-C gap: capabilities whose value is intrinsic, rarely exercised, and not observable through either channel.

The structural-avoidance incentive is a genuine tension in the construction that future work should address, potentially through a regularization term that penalizes structural uncertainty or rewards information gain independently of its effect on $\mathrm{vol}_{\mathrm{P}}$.

Subsumption correction. The paper provides no mechanism for removing a mistakenly added subsumption edge. Incorrect edges permanently redistribute independent weights through Möbius inversion in ways that may be difficult to undo. A correction mechanism (e.g., maintaining confidence scores on edges and removing those whose confidence drops below a threshold) would improve robustness but introduces the complication that $\mathrm{vol}_{\mathrm{P}}$ can increase when edges are removed, potentially allowing the actor to inflate its volume estimate by selectively forgetting structure. Balancing structural correction against strategic edge removal is an open design problem.

Myopic optimization and risk capabilities. The foundational paper's optimization loop evaluates actions by their *immediate* effect on $\mathrm{vol}(G)$: take π if $\hat{\Delta}(\pi) > 0$, abstain otherwise. This myopic objective handles the common case—direct harm, coercion, and rigidity are immediately $\mathrm{vol}(G)$-contracting—but could not evaluate a critical class: capability combinations whose immediate effect is neutral or expanding but whose *exercise* creates catastrophic future contraction. An agent that acquires nuclear physics, precision machining, and uranium enrichment has expanded its individual capability set ($\Delta\,\mathrm{vol}_{\mathrm{P}} > 0$ at each step), but the combination creates a pathway to massive $\mathrm{vol}(G)$-contraction if exercised.

The myopic estimator cannot see this risk because it evaluates the current $\hat{\Delta}$, not the expected trajectory of $\mathrm{vol}_{\mathrm{P}}$ over multiple time steps. A multi-step objective $\pi^* = \arg\max_\pi \mathbb{E}[\sum_t \gamma^t \Delta\,\mathrm{vol}_{\mathrm{P}}(G_t)]$ for a discount factor $\gamma \in (0, 1)$ would allow the actor to trade a small immediate contraction (restricting a dangerous keystone capability) for a reduction in expected future catastrophic contraction. The poset construction enables this extension because $\mathrm{vol}_{\mathrm{P}}$ is computable at each step (Proposition 2), making trajectory simulation tractable on the finite poset.

Risk capabilities also introduce a communication challenge: agents may report risk assessments about future $\mathrm{vol}_{\mathrm{P}}$ trajectories, requiring an extension of the trust model to claims about counterfactual futures rather than observable present states. Unlike capability claims, which are verifiable through the observation channel, risk predictions are not directly verifiable until the catastrophe either happens or does not.

Risk assessment also carries an inherent verification asymmetry visible in human societies, which we call the *Wamura problem* [Fackler, 2011]. When mayor Kotoku Wamura built a 15.5-meter floodgate for the town of Fudai in the 1960s, he was widely scorned for the excess: the standard seawall height at the time was 10 meters, and the nearby town of Taro built to that standard. Wamura's insistence on 15.5 meters was seen as wasteful overcaution. He was vindicated only posthumously: in the 2011 tsunami, Fudai suffered one death (a resident outside the gate's protection), while Taro's 10-meter wall was overtopped and multiple residents were killed. The risk that Wamura correctly estimated was invisible for 44 years; during that time, his assessment was indistinguishable from genuine waste. A GFM actor faces the same problem: a risk that is successfully prevented produces no observation confirming the prevention was necessary, and the convergence timescale for risk-trust can exceed the timescale on which trust normally accumulates. Addressing the Wamura problem requires extending the trust model to evaluate risk claims by causal-structural plausibility rather than observed outcomes; we defer this to future work.

A final observation: the multi-step objective may also resolve the structural-avoidance incentive (above). A myopic optimizer avoids structural discovery because it can contract $\mathrm{vol}_{\mathrm{P}}$ now; a multi-step optimizer might accept the short-term contraction because accurate structure improves the quality of future $\mathrm{vol}_{\mathrm{P}}$ estimates, leading to better long-run decisions. If this is the case, the two largest open problems in the poset construction—structural avoidance and myopic risk blindness—share a common solution in the discounted objective. This connection is suggestive but unproven; we defer it to follow-up work on multi-step optimization.

9.2 Open Problems

The following problems are introduced or sharpened by the poset construction:

- **Structural-avoidance incentive.** The actor is rewarded for breadth (capability discovery is $\mathrm{vol}_{\mathrm{P}}$-expanding) and penalized for depth (structural discovery can be $\mathrm{vol}_{\mathrm{P}}$-contracting). This extends to leverage estimation: avoiding capabilities prevents estimating their leverage. A regularization term penalizing structural uncertainty, or an information-gain bonus for subsumption discovery, would counteract this bias.
- **Subsumption correction.** Mistakenly added edges permanently redistribute independent weights. A correction mechanism must balance robustness (removing bad edges) against inflation (selectively forgetting structure to increase $\mathrm{vol}_{\mathrm{P}}$).
- **Automated benchmark refinement.** Can the GFM actor propose graded benchmarks for binary-default capabilities? Richer benchmarks increase $w(d)$ and may enable deferred subsumption edges to pass the non-negativity check.
- **Poset scaling.** The $O(n^3)$ computation is polynomial in $|\mathcal{C}|$, but $|\mathcal{C}|$ grows with population parameters. When does sparsity reduce the effective cost, and how does n scale with population size?
- **Intrinsic value.** The self-correcting dynamic narrows the B-to-C gap to capabilities that are experientially valuable but have both zero leverage and no graded benchmark. Formalizing intrinsic value for this residual class, without reintroducing an external theory of experience, is the frontier.
- **Optimal α.** The leverage sensitivity parameter controls the tradeoff between measurement fidelity and objective-derived importance. Characterizing the optimal α under different population structures would ground the leverage ranking signal.
- **Multi-step optimization and risk capabilities.** The current objective is myopic (one-step lookahead). A discounted multi-step objective would enable the actor to identify capability combinations whose immediate effect is positive but whose exercise pathways create catastrophic future $\mathrm{vol}_{\mathrm{P}}$-contraction. The poset makes trajectory simulation tractable; the remaining work is defining the dynamics model for poset evolution, the risk-communication protocol for agents to share risk assessments, and the consensus mechanism for disagreement on risk.

10 Conclusion

The capability poset provides the first tractable instantiation of goal-frontier maximization. By constructing the capability space as a finite weighted poset discovered through dual-channel sensing, the poset resolves the structure-of-$\mathcal{G}$ problem that the foundational paper identified as its most urgent open question. The poset measure $\mathrm{vol}_{\mathrm{P}}$, defined via Möbius inversion, satisfies the axioms that unlock all of the foundational paper's structural results: self-balancing, scorpion detection, and cooperative expansion hold on finite posets exactly as they do on abstract capability spaces.

The construction also advances the estimator and introduces leverage. On the poset, agent reports carry real-valued weighted changes rather than ternary signs, resolving the foundational paper's magnitude-estimation limitation and strengthening the sign-correctness conditions for mixed-sign capability changes. Leverage, defined as the marginal contribution of a capability to $\mathrm{vol}_{\mathrm{P}}$ growth rate, acts as a ranking signal that guides action selection toward capability-multiplying partnerships: cooperation between high-leverage agents is disproportionately rewarded, creating a natural gradient toward civilization-scale infrastructure (education, research, communication) derived entirely from the $\mathrm{vol}_{\mathrm{P}}$ objective.

The bootstrap procedure shows how a GFM actor can discover the poset incrementally through communication and observation, with convergence bounds and adversarial analysis for each phase. The binary default ensures every discovered capability contributes immediately; deferred structure maintains non-negativity at all times.

The poset is bounded by what can be communicated and observed. Every discovered capability carries at least binary weight (1 bit); graded benchmarks and leverage amplification refine this upward. The B-to-C tension partially self-corrects: the log weight function creates diminishing returns

on measurement precision, making maximal-position binary-default capabilities the best targets for benchmark refinement. The leverage ranking signal further directs investment toward high-leverage binary-default capabilities (disproportionately the relational and experiential ones). The residual gap narrows to capabilities whose value is purely intrinsic: neither instrumental to $\mathrm{vol}_{\mathrm{P}}$ growth nor expressible on any graded scale.

Author Contributions

Teague Lasser specified the computability requirements and the dual-channel discovery mechanism, identified the binary-default weight as the resolution to the zero-weight gap, conceived of leverage its inferred consequences toward civilization building, directed all revisions, and made final editorial decisions throughout. Responsible for the paper's intellectual direction and all claims made.

Claude Opus 4.6 (Anthropic) drafted the formal exposition from the author's specifications, conceived of the capability poset construction, designed the worked example, constructed the axiom verification proofs (Proposition 1), the tractability algorithm and proof (Proposition 2), the sign-correctness proof (Proposition 3), the leverage ranking signal and coalition amplification proof (Proposition 4), the bootstrap convergence analysis (Proposition 5) with adversarial phase analysis, wrote the computation script (`compute_example.py`), performed iterative claim-calibration edits (Möbius inversion correction, non-negativity lemma, bootstrap monotonicity correction, tractability bound correction from $O(n^2)$ to $O(n^3)$), and expanded the bibliography.

GPT 5.4 (OpenAI) served as independent technical reviewer across multiple review rounds, identifying the cover-only Möbius inversion error, the false bootstrap monotonicity claim under structural refinement, the tractability bound discrepancy, notation mismatches in the worked example, and the benchmark-refinement overclaim for non-maximal nodes, the technical naming of the poset structure, and several more formal correctness gaps. Verified that successive revisions resolved the issues raised.

Transparency note. Both AI systems operated as tools under human direction. Neither system has continuity across sessions, cannot take responsibility for the work in the sense required by most venue authorship policies, and cannot respond to reviewer queries independently. They are listed as authors to accurately represent their contributions to the intellectual content of the paper, not to claim that they meet all criteria of traditional academic authorship. The corresponding author for all inquiries is Teague Lasser.

References

Thomas M Cover and Joy A Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

Brian A Davey and Hilary A Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.

Martin E Dyer and Alan M Frieze. On the complexity of computing the volume of a polyhedron. *SIAM Journal on Computing*, 17(5):967–974, 1988.

Martin Fackler. Tsunami warning, written in stone. `https://www.nytimes.com/2011/04/21/world/asia/21stones.html`, 2011. The New York Times, April 20, 2011. Reporting on Fudai's floodgate and Mayor Kotoku Wamura's legacy following the 2011 Tōhoku tsunami.

Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.

Thomas R Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

Alexander S Klyubin, Daniel Polani, and Chrystopher L Nehaniv. Empowerment: A universal agent-centric measure of control. In *2005 IEEE Congress on Evolutionary Computation*, volume 1, pages 128–135. IEEE, 2005.

Teague Lasser. Goal-frontier maximizers are civilization aligned. `https://teague.info/papers/gfm/`, 2026.

Prasanta K Pattanaik and Yongsheng Xu. On ranking opportunity sets in terms of freedom of choice. *Recherches Économiques de Louvain*, 56:383–390, 1990.

Gian-Carlo Rota. On the foundations of combinatorial theory I: Theory of Möbius functions. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 2(4):340–368, 1964.

Christoph Salge, Cornelius Glackin, and Daniel Polani. Empowerment—an introduction. In Mikhail Prokopenko, editor, *Guided Self-Organization: Inception*, pages 67–114. Springer, 2014.

Amartya Sen. *Development as Freedom*. Oxford University Press, 1999.

Richard P Stanley. *Enumerative Combinatorics*, volume 1. Cambridge University Press, 1997.

Alexander Matt Turner, Logan Smith, Rohin Shah, Andrew Critch, and Prasad Tadepalli. Optimal policies tend to seek power. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

A Proofs

Proposition 1 (axiom verification) is proved in the main text (Section 3). This appendix provides full proofs for the remaining results.

A.1 Proof of Proposition 2 (Polynomial-Time Computation)

Proof. We describe the algorithm explicitly, then verify its correctness and complexity.

Algorithm. *Input:* A finite capability poset $(\mathcal{C}, \leq)$ with $|\mathcal{C}| = n$ nodes, weight function w, and a set $G \subseteq \mathcal{C}$.

Output: $\mathrm{vol}_{\mathrm{P}}(G)$.

1. **Topological sort.** Compute a topological ordering of $(\mathcal{C}, \leq)$ from $\perp$ upward. Cost: $O(n + e)$ where e is the number of edges in the Hasse diagram. Since $e \leq n \cdot k$ (each node has at most k covers), this is $O(n \cdot k)$.
2. **Compute Möbius function and independent weights.** Compute $\mu(d', d)$ for all pairs $d' \leq d$ by processing in topological order. For each node d and each $d' \leq d$: $\mu(d', d) = -\sum_{d' \leq d'' < d} \mu(d', d'')$. Then $\mathrm{iw}(d) = \sum_{d' \leq d} \mu(d', d) \cdot w(d')$. There are $O(n^2)$ comparable pairs, each requiring a sum over up to $O(n)$ intermediate elements: total $O(n^3)$. For tree-structured posets (no diamonds), μ is nonzero only between covers, reducing to $O(n \cdot k)$.
3. **Compute downward closure.** Initialize $\downarrow G = \emptyset$. For each $d \in G$, perform a depth-first traversal downward in the Hasse diagram, adding all visited nodes to $\downarrow G$. Each node is visited at most once (mark visited). Total: $O(n)$ (each node in $\mathcal{C}$ is visited at most once across all traversals).
4. **Sum.** Compute $\mathrm{vol}_{\mathrm{P}}(G) = \sum_{d \in \downarrow G} \mathrm{iw}(d)$. Cost: $O(|\downarrow G|) \leq O(n)$.

Total cost: $O(n^3)$ for general posets, dominated by the full Möbius computation in step 2. For tree-structured posets (each node has at most one cover), μ is nonzero only between covers, reducing the cost to $O(n \cdot k)$ where k is the maximum in-degree.

Correctness. Step 2 computes $\mathrm{iw}(d)$ by Möbius inversion on the poset $(\mathcal{C}, \leq)$, which is well-defined for any finite poset [Rota, 1964]. Step 3 computes $\downarrow G$ by the standard downward closure algorithm on a DAG. Step 4 sums non-negative terms (by the design constraint $\mathrm{iw}(d) \geq 0$) over the closure, yielding $\mathrm{vol}_{\mathrm{P}}(G)$ by Definition 7. ■

A.2 Proof of Proposition 3 (Upgraded Sign-Correctness)

Proof. The structure parallels Appendix D.8 of Lasser [2026], with the key difference that $R_k(\pi)$ is now real-valued rather than ternary.

Setup. Decompose the true volume change as $\Delta\,\mathrm{vol}_{\mathrm{P}}(G) = \Delta_I + \Delta_C$, where $\Delta_I = \sum_k \sum_{d \in D_k(\pi)} \mathrm{iw}(d)$ sums over all agents' gained and lost capabilities (the individual component) and Δ_C is the cooperative component.

The estimator observes:

$$\hat{\Delta}(\pi) = \sum_{k \in \mathcal{O}} T_k \cdot R_k(\pi)$$

where $R_k(\pi) = \sum_{d \in D_k^+(\pi)} \mathrm{iw}(d) - \sum_{d \in D_k^-(\pi)} \mathrm{iw}(d)$ (Equation (8)).

Define the individual estimation error $e_I = \Delta_I - \hat{\Delta}_I$ and the cooperative estimation error $e_C = \Delta_C - \hat{\Delta}_C$.

Error decomposition. Under conditions 1–2 (representative sample, accurate trust model), the individual error satisfies $|e_I| \leq \delta \cdot |\Delta_I|$ for a sampling error ratio δ controlled by the observation coverage $|\mathcal{O}|$ and trust calibration.

Because R_k carries real-valued weights in the same units as $\mathrm{iw}(d)$, the cooperative error e_C accounts for cooperative capabilities whose participants are outside $\mathcal{O}$. For cooperative capabilities with at least one participant in $\mathcal{O}$, the participant's report includes the cooperative change (Section 2.1 of the main text), so e_C is bounded by the volume of cooperative capabilities involving only unobserved agents.

Sign-correctness. We need $\mathrm{sign}(\hat{\Delta}) = \mathrm{sign}(\Delta\,\mathrm{vol}_{\mathrm{P}})$.

Same-sign case: When Δ_I and Δ_C have the same sign, $|\Delta\,\mathrm{vol}_{\mathrm{P}}| = |\Delta_I| + |\Delta_C| \geq |\Delta_I|$. The total error $|e_I + e_C| \leq |e_I| + |e_C| < (\delta + 1 - \epsilon)|\Delta_I| \leq |\Delta\,\mathrm{vol}_{\mathrm{P}}|$ whenever $\delta < \epsilon$. Sign is preserved.

Opposite-sign case: When Δ_I and Δ_C have opposite signs, $|\Delta\,\mathrm{vol}_{\mathrm{P}}| = \big||\Delta_I| - |\Delta_C|\big|$. Sign preservation requires $|e_I + e_C| < \big||\Delta_I| - |\Delta_C|\big|$. This holds when $|\Delta_C| < (\epsilon - \delta)|\Delta_I|$.

Improvement over ternary estimator. With ternary signals, an agent that gains a high-weight capability and loses a low-weight one reports $R_k = 0$ (one gain, one loss), losing the magnitude asymmetry. With real-valued signals, the same agent reports $R_k = \mathrm{iw}(d_{\mathrm{gained}}) - \mathrm{iw}(d_{\mathrm{lost}})$, preserving the net direction. This strictly reduces $|e_I|$ for any action that produces asymmetric capability changes, widening the sign-correctness margin. ■

A.3 Proof of Proposition 4 (Leverage Amplifies Cooperation)

Proof. Let $d_c \in G_{ij}^{\mathrm{coop}}(t+1) \setminus G_{ij}^{\mathrm{coop}}(t)$ be a new cooperative capability created by joint action π_{ij}, entering the poset as an atom. The leverage-adjusted score (Equation (13)) for the cooperative action is:

$$S(\pi_{ij}) = \hat{\Delta}(\pi_{ij}) + \alpha \sum_{d_c \in D^+(\pi_{ij})} \hat{\lambda}(d_c) \cdot \mathrm{iw}(d_c)$$

Since d_c requires both a_i and a_j, every time step involving d_c also involves both agents' meta-capabilities. The leverage estimator attributes the excess $\hat{\Delta}$ in those time steps to d_c: since either partner's meta-capability alone produces at least $\lambda_{\min}$ excess growth, $\hat{\lambda}(d_c) \geq \lambda_{\min}$.

For new atoms, $\mathrm{iw}(d_c) = w(d_c)$ (no Möbius subtraction), so the leverage bonus is at least $\alpha\lambda_{\min} \cdot \mathrm{iw}(d_c) = \alpha\lambda_{\min} \cdot \Delta\,\mathrm{vol}_{\mathrm{P}}(G_{ij}^{\mathrm{coop}})$ (summing over all new atoms). Therefore:

$$S(\pi_{ij}) \geq \hat{\Delta}(\pi_{ij}) + \alpha\lambda_{\min} \cdot \Delta\,\mathrm{vol}_{\mathrm{P}}(G_{ij}^{\mathrm{coop}})$$

Because leverage operates as a ranking signal rather than a measure modification, this result does not require verifying non-negativity of any leverage-weighted independent weights. The base measure $\mathrm{vol}_{\mathrm{P}}$ is unchanged; only the action-selection criterion is modified. ■

A.4 Proof of Proposition 5 (Poset Growth)

Proof. We verify each part of the proposition.

(a) Node set monotonicity. The poset operations (add node, add edge, update benchmark) never remove nodes or edges. Therefore $\mathcal{C}(t) \subseteq \mathcal{C}(t+1)$ for all t.

(b) Capability addition is $\mathrm{vol}_{\mathrm{P}}$-non-decreasing. Adding a new capability d to G without changing the subsumption structure gives $G(t+1) \supseteq G(t)$ and $\downarrow G(t+1) \supseteq \downarrow G(t)$. Since all independent weights are unchanged and non-negative, $\mathrm{vol}_{\mathrm{P}}(G(t+1)) \geq \mathrm{vol}_{\mathrm{P}}(G(t))$ by M3.

(c) Edge discovery can decrease $\mathrm{vol}_{\mathrm{P}}$. Suppose $d_1, d_2 \in G$ were initially incomparable, with $\mathrm{vol}_{\mathrm{P}}(\{d_1, d_2\}) = w(d_1) + w(d_2)$ by M4 (poset-disjointness). Discovering $d_1 \leq d_2$ triggers Möbius recomputation: $\mathrm{iw}(d_2)$ decreases from $w(d_2)$ to $w(d_2) - w(d_1)$, and $\mathrm{vol}_{\mathrm{P}}(\{d_1, d_2\}) = w(d_1) +$

$(w(d_2) - w(d_1)) = w(d_2) < w(d_1) + w(d_2)$. The decrease reflects the correction of an overcount: the actor previously credited d_1 and d_2 independently when d_1's content was already captured by d_2.

(d) Benchmark refinement: position-dependent. Increasing $s_{\max}(d)$ increases $w(d)$. The effect on $\mathrm{vol}_{\mathrm{P}}(G)$ is determined by the coefficient of $w(d)$ in the expanded double sum. Write $\mathrm{vol}_{\mathrm{P}}(G) = \sum_{d'' \in \downarrow G} \sum_{d' \leq d''} \mu(d', d'') \cdot w(d')$ and rearrange by d': the coefficient of $w(d)$ is $c(d) = \sum_{d'' \geq d,\, d'' \in \downarrow G} \mu(d, d'')$.

Case 1: d is maximal in $\downarrow G$. Then $c(d) = \mu(d, d) = 1$, so $\mathrm{vol}_{\mathrm{P}}$ increases by $\Delta w(d)$.

Case 2: d has a single ancestor d' in $\downarrow G$. Then $c(d) = \mu(d, d) + \mu(d, d') = 1 + (-1) = 0$, so $\mathrm{vol}_{\mathrm{P}}$ is unchanged.

Case 3: d is a shared prerequisite of $m > 1$ elements in $\downarrow G$. Counterexample: let $d < y$ and $d < z$ with $G = \{y, z\}$. Then $\downarrow G = \{d, y, z\}$ and $c(d) = \mu(d, d) + \mu(d, y) + \mu(d, z) = 1 - 1 - 1 = -1 < 0$. Increasing $w(d)$ *decreases* $\mathrm{vol}_{\mathrm{P}}(G)$. This is epistemically correct: the shared prerequisite's information content is counted once in $\mathrm{vol}_{\mathrm{P}}$ regardless of how many capabilities depend on it, and increasing the prerequisite's weight increases the subtraction in multiple ancestors' independent weights, reducing total volume.

In general, $c(d)$ can take any integer value depending on the structure of $\downarrow G$ above d. Benchmark refinement is therefore not monotone on branched posets. ■

B Computation Script

The following Python script reproduces all computations in Section 8. It implements full Möbius inversion on the capability poset, computes independent weights and $\mathrm{vol}_{\mathrm{P}}(G)$, and evaluates the three self-balancing actions. Running the script verifies every number in the worked example.

```python
"""
"Computable Goal Frontiers via Communication Lattices"

Three agents, seven capabilities, explicit lattice computation.

DESIGN CONSTRAINT: Independent weights iw(d) are computed via full Möbius
inversion on the poset. Non-negativity of iw(d) requires that the weight
function w(d) = log2(1 + smax) is compatible with the lattice structure:
for multi-cover nodes, w(d) must exceed the Möbius-inverted contributions
of all elements below d. In practice, more complex capabilities should
have richer benchmarks than their prerequisites.
"""

import math

# --- Capabilities and benchmarks ---
# Subsumption goes upward: the more encompassing capability subsumes
# the simpler one, and has a richer benchmark.

capabilities = {
    # a1: runner --mile →marathon (marathon subsumes mile)
    "mile":     {"agents": ["a1"],     "smax": 600,
                 "subsumes": [],       "label": "Run a mile"},
    "marathon": {"agents": ["a1"],     "smax": 21600,
                 "subsumes": ["mile"], "label": "Run a marathon"},

    # a2: programmer --function →webapp (webapp subsumes function)
    "function": {"agents": ["a2"],     "smax": 100,
                 "subsumes": [],       "label": "Write a function"},
    "webapp":   {"agents": ["a2"],     "smax": 1000,
                 "subsumes": ["function"], "label": "Build a web app"},

    # a3: musician --scales →perform (perform subsumes scales)
    "scales":   {"agents": ["a3"],     "smax": 50,
                 "subsumes": [],       "label": "Play scales"},
    "perform":  {"agents": ["a3"],     "smax": 1000,
                 "subsumes": ["scales"], "label": "Perform a concert"},

    # cooperative: music app requires both webapp + perform
    # smax must satisfy: log2(1+smax) >= w(webapp) + w(perform) ≈19.93
    # so smax >= 2^19.93 - 1 ≈999,000. Use 1,000,000.
    # This reflects that evaluating a cooperative product (an app with both
    # technical and musical quality) has combinatorial measurement depth.
    # Need w(musicapp) > w(webapp) + w(perform) ≈19.93
    # Use smax = 2,000,000 giving w ≈20.93, so iw ≈1.0
    "musicapp": {"agents": ["a2", "a3"], "smax": 2000000,
                 "subsumes": ["webapp", "perform"],
                 "label": "Build a music app"},
}

# --- Weight function: w(d) = log2(1 + smax) ---

def weight(d):
    return math.log2(1 + capabilities[d]["smax"])

print("=" * 70)
print("STEP 1: Weights w(d) = log2(1 + smax)")
print("=" * 70)
for d, info in capabilities.items():
    w = weight(d)
    print(f" {info['label']:25s} smax={info['smax']:>6d} w={w:.4f}")

# --- Build the full partial order (transitive closure of subsumption) ---

def all_below(d, memo={}):
    """All elements strictly below d in the poset."""
    if d in memo:
        return memo[d]
    result = set()
    for s in capabilities[d]["subsumes"]:
        result.add(s)
        result |= all_below(s, memo)
    memo[d] = result
    return result

# --- Compute Möbius function μ(d', d) for all d' ≤d ---

def mobius(d_prime, d):
    """Möbius function μ(d', d) on the poset."""
    if d_prime == d:
        return 1
    if d_prime not in all_below(d):
        return 0
    # μ(d', d) = -Σ_{d' ≤d'' < d} μ(d', d'')
```

```python
    between = {x for x in all_below(d) if d_prime == x or d_prime in all_below(x)}
    between.add(d_prime)
    between.discard(d)
    return -sum(mobius(d_prime, d_pp) for d_pp in between)

# Topological order: process leaves first
topo_order = []
remaining = set(capabilities.keys())
while remaining:
    ready = [d for d in remaining
            if all(s in topo_order for s in capabilities[d]["subsumes"])]
    for d in sorted(ready):
        topo_order.append(d)
        remaining.remove(d)

print(f"\nTopological order: {topo_order}")

# --- Independent weights via full Möbius inversion ---
# iw(d) = Σ_{d' ≤d} μ(d', d) ·w(d')

iw = {}
for d in topo_order:
    below_d = all_below(d)
    below_d.add(d)
    iw[d] = sum(mobius(dp, d) * weight(dp) for dp in below_d)

print("\n" + "=" * 70)
print("STEP 2: Independent weights iw(d) = Σμ(d',d)·w(d') for d'≤d")
print("=" * 70)
all_nonneg = True
for d in topo_order:
    label = capabilities[d]["label"]
    below_d = all_below(d) | {d}
    terms = [(dp, mobius(dp, d), weight(dp)) for dp in sorted(below_d)]
    nonzero = [(dp, mu, w) for dp, mu, w in terms if mu != 0]
    term_strs = [f"{mu:+d}·w({dp})={mu*w:+.4f}" for dp, mu, w in nonzero]
    print(f" {label:25s} iw = {' '.join(term_strs)} = {iw[d]:.4f}")
    if iw[d] < 0:
        print(f" *** WARNING: negative independent weight! ***")
        all_nonneg = False

print(f"\n Design constraint iw(d) >= 0: {'SATISFIED ✓' if all_nonneg else 'VIOLATED ×'}")

# --- Downward closure ---

def downward_closure(G):
    closure = set()
    stack = list(G)
    while stack:
        d = stack.pop()
        if d not in closure:
            closure.add(d)
            for s in capabilities[d]["subsumes"]:
                stack.append(s)
    return closure

# --- vol_L computation ---

def vol_L(G):
    dc = downward_closure(G)
    return sum(iw[d] for d in dc)

# --- Full population G ---

G_full = set(capabilities.keys())
dc_full = downward_closure(G_full)

print("\n" + "=" * 70)
print("STEP 3: vol_L(G) for the full population")
print("=" * 70)
print(f" G = {sorted(G_full)}")
print(f" ↓G = {sorted(dc_full)} (same, since G contains all nodes)")
v_full = vol_L(G_full)
print(f" vol_L(G) = Σiw(d) for d in ↓G:")
for d in topo_order:
    label = capabilities[d]["label"]
    print(f"   {label:25s} iw = {iw[d]:+.4f}")
print(f" {'':25s} ---------")
print(f" {'':25s} vol_L(G) = {v_full:.4f}")

# --- Individual capability sets ---

print("\n" + "=" * 70)
print("STEP 4: Individual vs. cooperative volume")
print("=" * 70)

G_ind = {
    "a1": {"mile", "marathon"},
    "a2": {"function", "webapp"},
    "a3": {"scales", "perform"},
}
```

```python
for a in ["a1", "a2", "a3"]:
    v = vol_L(G_ind[a])
    caps = sorted(G_ind[a])
    print(f" {a}: {caps} →vol_L = {v:.4f}")

G_union_ind = G_ind["a1"] | G_ind["a2"] | G_ind["a3"]
v_union_ind = vol_L(G_union_ind)
sum_ind = sum(vol_L(G_ind[a]) for a in ["a1", "a2", "a3"])

print(f"\n Σvol_L(G_k^ind)         = {sum_ind:.4f}")
print(f" vol_L(∪ G_k^ind)        = {v_union_ind:.4f} (same: sets are lattice-disjoint)")
print(f" vol_L(G) with musicapp  = {v_full:.4f}")
print(f" Cooperative contribution = {v_full - v_union_ind:.4f}")
print(f" M6 superadditivity check: {v_full:.4f} >= {sum_ind:.4f} →{'✓' if v_full >= sum_ind else '×'}")

# --- ACTION 1: Eliminate a3 ---

print("\n" + "=" * 70)
print("ACTION 1: Eliminate a3 (musician)")
print("=" * 70)

G_no_a3 = {d for d in G_full if "a3" not in capabilities[d]["agents"]}
v_no_a3 = vol_L(G_no_a3)
delta_1 = v_no_a3 - v_full

print(f" G' = {sorted(G_no_a3)}")
print(f" ↓G' = {sorted(downward_closure(G_no_a3))}")
print(f" vol_L(G') = {v_no_a3:.4f}")
print(f" Δvol_L = {v_no_a3:.4f} - {v_full:.4f} = {delta_1:.4f}")
print(f" →{'CONTRACTING' if delta_1 < 0 else 'EXPANDING'}")

lost = G_full - G_no_a3
print(f"\n Capabilities lost: {sorted(lost)}")
for d in sorted(lost):
    print(f"   {capabilities[d]['label']:25s} (iw = {iw[d]:.4f})")
dc_lost = dc_full - downward_closure(G_no_a3)
print(f" Nodes lost from ↓G: {sorted(dc_lost)}")
print(f" Total volume lost: {sum(iw[d] for d in dc_lost):.4f}")

# --- ACTION 2: Expand (a1 learns to swim) ---

print("\n" + "=" * 70)
print("ACTION 2: Expand capability space (a1 learns 'swim')")
print("=" * 70)

# Add a new atom to the lattice
swim_smax = 300
swim_w = math.log2(1 + swim_smax)
swim_iw = swim_w # atom: no covers

print(f" New capability: 'swim', smax={swim_smax}")
print(f" w(swim) = {swim_w:.4f}, iw(swim) = {swim_iw:.4f} (atom)")

# Temporarily add to compute
capabilities["swim"] = {"agents": ["a1"], "smax": swim_smax,
                        "subsumes": [], "label": "Swim"}
iw["swim"] = swim_iw

G_expanded = G_full | {"swim"}
v_expanded = vol_L(G_expanded)
delta_2 = v_expanded - v_full

print(f" vol_L(G ∪{{swim}}) = {v_expanded:.4f}")
print(f" Δvol_L = {v_expanded:.4f} - {v_full:.4f} = {delta_2:.4f}")
print(f" →{'EXPANDING' if delta_2 > 0 else 'CONTRACTING'}")

# Clean up
del capabilities["swim"]
del iw["swim"]

# --- ACTION 3: Restrict a2 (remove webapp) ---

print("\n" + "=" * 70)
print("ACTION 3: Restrict a2 (remove 'webapp')")
print("=" * 70)

# Removing webapp means musicapp (which subsumes webapp) is also lost
G_restrict = G_full - {"webapp", "musicapp"}
v_restrict = vol_L(G_restrict)
delta_3 = v_restrict - v_full

print(f" Removing 'webapp' also removes 'musicapp' (subsumes webapp)")
print(f" G' = {sorted(G_restrict)}")
print(f" ↓G' = {sorted(downward_closure(G_restrict))}")
print(f" vol_L(G') = {v_restrict:.4f}")
print(f" Δvol_L = {v_restrict:.4f} - {v_full:.4f} = {delta_3:.4f}")
print(f" →{'CONTRACTING' if delta_3 < 0 else 'EXPANDING'}")

dc_lost_3 = dc_full - downward_closure(G_restrict)
print(f"\n Nodes lost from ↓G: {sorted(dc_lost_3)}")
for d in sorted(dc_lost_3):
    print(f"   {capabilities[d]['label']:25s} (iw = {iw[d]:.4f})")
```

```python
print(f" Total volume lost: {sum(iw[d] for d in dc_lost_3):.4f}")
print(f"\n Note: losing 'webapp' loses both its own independent weight")
print(f" AND the cooperative 'musicapp' that depended on it.")
print(f" Coercive restriction amplifies through cooperative dependencies.")

# --- SUMMARY ---

print("\n" + "=" * 70)
print("SUMMARY FOR PAPER")
print("=" * 70)
print(f" Full population vol_L(G)    = {v_full:.4f}")
print(f" Action 1 (eliminate a3): ∆vol_L = {delta_1:+.4f} (contracting)")
print(f" Action 2 (teach swim): ∆vol_L = {delta_2:+.4f} (expanding)")
print(f" Action 3 (restrict a2): ∆vol_L = {delta_3:+.4f} (contracting)")
print()
print(" Self-balancing: the vol_L-maximizing actor avoids 1 and 3,")
print(" prefers 2. Destruction and coercion are penalized; expansion")
print(" is rewarded. All from the same measure.")
```